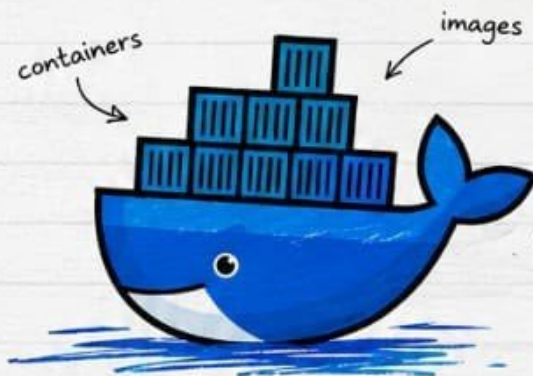


VERIQTA

- ✓ Engine
- ✓ Runtime
- ✓ Storage
- ✓ Networking
- ✓ Security
- ✓ Performance
- ✓ Build
- ✓ Troubleshooting



From the kernel
to the cloud.
Understand.
Optimize.
Master Docker

anywhere



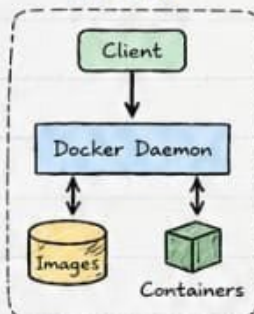
DOCKER INTERNALS

ENGINEER'S NOTEBOOK

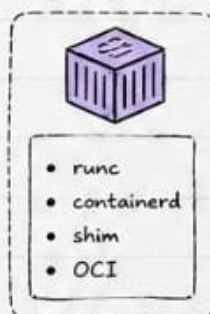
Handwritten Notes for DevOps & Platform Engineers

UNDERSTAND DEEPLY • TROUBLESHOOT FASTER • BUILD BETTER

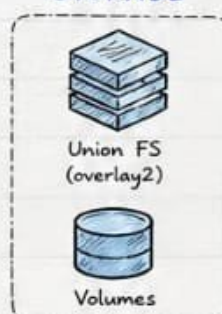
ENGINE



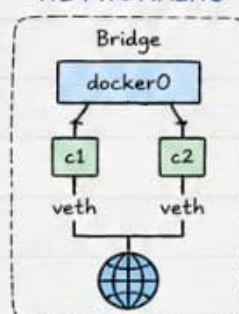
RUNTIME



STORAGE



NETWORKING



SECURITY



Why Docker Internals?

- ✓ Solve real-world problems
- ✓ Optimize performance
- ✓ Design robust systems
- ✓ Master container technology

```
$ docker run -it --rm alpine sh
# ls /
bin  dev  etc  home  proc  usr
lib  sys  tmp  var
# You are in a container
You are in control.
Understand the internals.
Own the platform.
```

Kernel

- Namespaces
- Cgroups
- Netfilter
- OverlayFS
- Capabilities



20 PAGES
COMPLETE GUIDE



DEEP DIVE
INTO DOCKER
INTERNALS



REAL DIAGRAMS
HANDWRITTEN
EXPLANATIONS



PRODUCTION
PROVEN
PRACTICES



ENGINEERED
FOR SUCCESS

≡ NOT JUST NOTES. A MASTERY SYSTEM. ≡



DOCKER ENGINE ARCHITECTURE



Understand how Docker works under the hood.

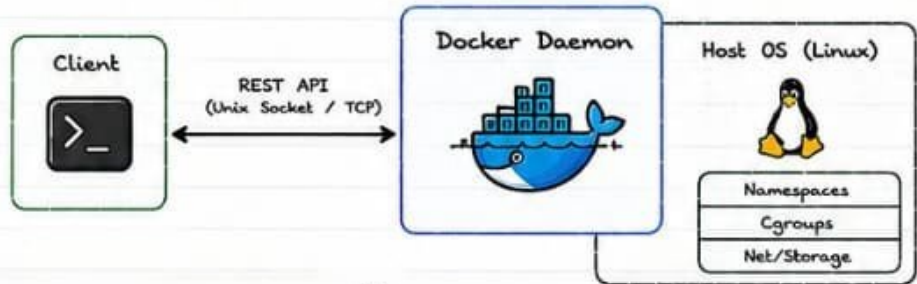
From your command → to running containers on Linux.

1. DOCKER ENGINE OVERVIEW

Docker Engine is a client-server application that builds, runs and manages containers.

It consists of:

- Docker Client
- Docker Daemon
- Docker REST API
- Docker Objects



2. DOCKER CLIENT



- The Docker CLI is the primary way users interact with Docker.
- It sends commands to the daemon using the Docker REST API.
- Examples:

```
docker run nginx
docker ps
docker build -t app .
docker push myapp:1.0
```

3. DOCKER DAEMON



- The daemon (dockerd) listens for API requests and manages Docker objects.
- It handles:
 - ✓ Building images
 - ✓ Running containers
 - ✓ Managing networks
 - ✓ Managing volumes
 - ✓ Managing plugins
- It talks to containerd to manage container lifecycles.

4. DOCKER REST API



- Docker exposes a RESTful API over:
 - ✓ Unix Socket: /var/run/docker.sock
 - ✓ TCP Socket : 2375 (insecure)
 - ✓ TCP TLS : 2376 (secure)
- ALL Docker commands are translated to REST API calls.
- You can interact with Docker using any HTTP client!

5. DOCKER OBJECTS



Image

Read-only template with instructions for creating a container.



Container

Running instance of an image. Has its own filesystem, network, runtime.



Volume

Persistent data store managed by Docker. Lives outside container lifecycle.



Network

Allows containers to communicate with each other and the outside world.



Image Layer

Read-only layer in a union filesystem. Layered for efficiency.



Plugin

Extends Docker functionality (e.g., network, storage, auth).

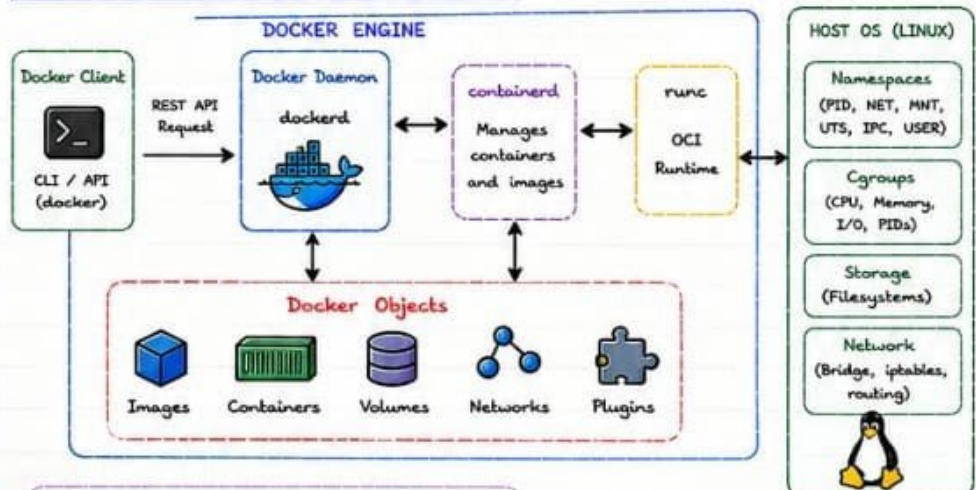
6. COMPLETE REQUEST FLOW

- 1 User runs a Docker command (e.g., docker run nginx)
- 2 Docker Client parses the command
- 3 Client sends REST API request to the Docker Daemon
- 4 Daemon authenticates & validates the request
- 5 Daemon performs the requested action (e.g., pull image, create container)
- 6 Daemon delegates container management to containerd
- 7 containerd interacts with runc to create and run the container
- 8 Container starts and is attached to the required resources (network, storage, cgroups, namespaces)
- 9 Daemon sends response back to the Client

KEY TAKEAWAYS

- ✓ You interact using the Docker Client.
- ✓ Commands are sent over the REST API.
- ✓ The Docker Daemon does the heavy lifting.
- ✓ Objects are managed by the Daemon.
- ✓ containerd + runc run the containers using Linux kernel features.

7. END-TO-END ARCHITECTURE DIAGRAM



EXAMPLE: docker run nginx

1. Client → Daemon (create container)
2. Daemon pulls nginx image (if not present)
3. Image stored as layered read-only files
4. Daemon creates container via containerd
5. runc starts the container process
6. Container gets IP, mounts, limits, etc.
7. Container is running! ✓

COMMON SOCKET LOCATION

Unix Socket (default):
/var/run/docker.sock

Check it with:
ls -l /var/run/docker.sock



Docker Engine is the brain.
Your commands are the instructions.
The Linux kernel does the heavy lifting.





OPEN CONTAINER INITIATIVE (OCI)

★ Industry standards for building, running and distributing containers. ★



OCI is an open governance structure for creating open industry standards around container formats and runtime.

OCI

1. WHAT IS OCI?

- Open Container Initiative (OCI) is a Linux Foundation project.
- It defines open specifications for container runtimes, images, and distribution.
- Its goal is to create a standard that any tool or vendor can implement.
- OCI does not provide an implementation. It defines the rules.
- Docker, containerd, Podman, CRI-O, Buildah, nerdctl, and many others follow OCI.

OCI GOAL

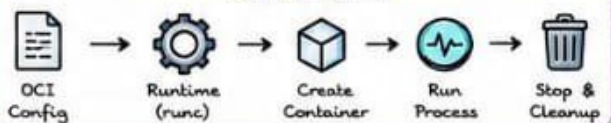


Create open, vendor-neutral standards that promote interoperability, innovation and choice.

2. OCI RUNTIME SPECIFICATION

- Defines how a container runtime should run containers.
- Specifies the lifecycle, configuration and interaction between runtime and container.
- Implemented by runtimes like runc, crun, youki, gVisor (OCI compatible).
- Ensures that any OCI compliant runtime can run containers created by any OCI tool.

RUNTIME FLOW



3. OCI IMAGE SPECIFICATION

- Defines the format of container images.
- Specifies image layout, manifests, layers, configs and metadata.
- Ensures images are portable across tools and registries.
- Implemented by tools like Buildah, Kaniko, Docker, containerd, img, oras.

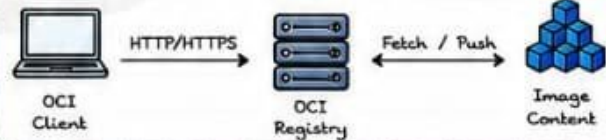
OCI IMAGE LAYOUT



4. OCI DISTRIBUTION SPECIFICATION

- Defines the protocol and API for distributing images.
- Specifies how registries store, fetch and serve images.
- Ensures that any OCI compatible registry can work with any OCI compatible client.
- Implemented by registries like Docker Registry, Harbor, Quay, GHCR, Amazon ECR.

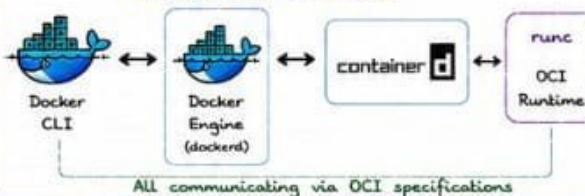
DISTRIBUTION FLOW



5. DOCKER AND OCI

- Docker fully embraces OCI specifications.
- Docker Engine uses containerd as the core runtime component.
- runc (OCI runtime) is used by containerd to run containers.
- Docker images follow OCI Image Spec.
- Docker Registry v2 follows OCI Distribution Spec.

DOCKER STACK (OCI BASED)



6. WHY OCI MATTERS

	Interoperability → Tools from different vendors can work together seamlessly.
	Vendor Neutrality → Avoid lock-in. You can switch tools without changing standards.
	Innovation → Lower barrier for new projects to build better container tools.
	Ecosystem Growth → Many tools, runtimes, registries and platforms can evolve together.
	Security & Reliability → Standardized formats and APIs improve audits, testing and trust.
	Future Proof → OCI is the foundation for modern cloud native and Kubernetes ecosystems.

KEY TAKEAWAYS



- ✓ OCI defines the standards.
- ✓ Docker, containerd, Podman implement them.
- ✓ Runtimes, images and registries all follow OCI.
- ✓ Open standards = freedom, portability, innovation.

THREE CORE OCI SPECIFICATIONS

- 1 **Runtime Spec** → How containers run
- 2 **Image Spec** → How container images are built and stored
- 3 **Distribution Spec** → How images are pushed and pulled

Remember

OCI is not a product. It's a standard. Standards build strong ecosystems!



Open standards. Open choices. Open future.





CONTAINERD

The core container runtime behind Docker and many cloud-native platforms.



It manages containers, images, snapshots and runs containers using OCI runtimes.



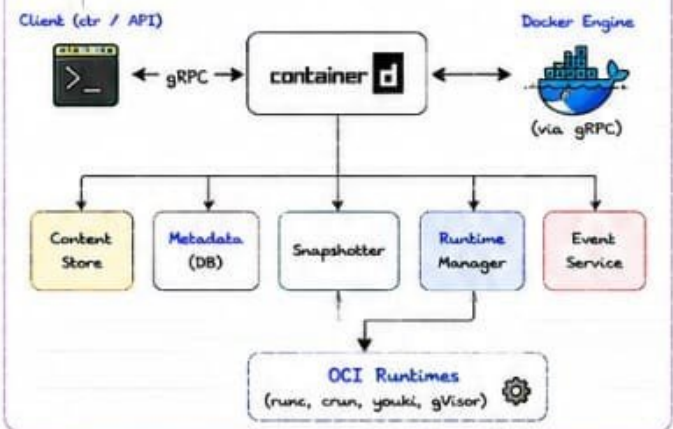
1. WHAT IS CONTAINERD?

- containerd is an industry-standard container runtime.
- It is an open source daemon that manages the complete container lifecycle.
- It is focused, lightweight and modular.
- Docker Engine uses containerd to run containers.
- Kubernetes (CRI) uses containerd as its default runtime.

KEY ROLES

- ✓ Pull, store and manage images
- ✓ Manage container lifecycle
- ✓ Manage storage (snapshots)
- ✓ Handle containers using OCI runtimes
- ✓ Publish events
- ✓ Provide gRPC API

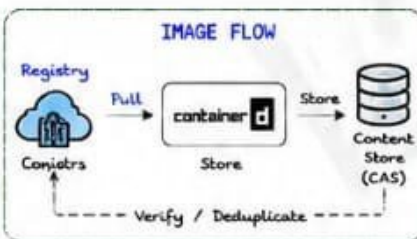
2. CONTAINERD ARCHITECTURE



3. IMAGE MANAGEMENT



- containerd pulls images from registries.
- Images are stored in the **content store**.
- Uses content-addressable storage (CAS).
- Handles image unpacking and references.
- Images are shared across namespaces.



4. RUNTIME MANAGEMENT



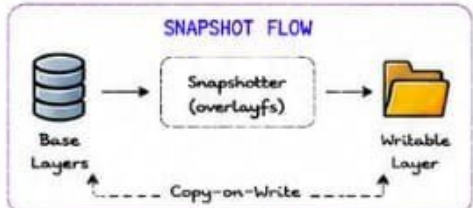
- Creates containers using OCI runtimes.
- Configures namespaces, cgroups, mounts, networking.
- Manages task lifecycle (create, start, stop, delete).
- Supports multiple runtimes.



5. SNAPSHOT MANAGEMENT



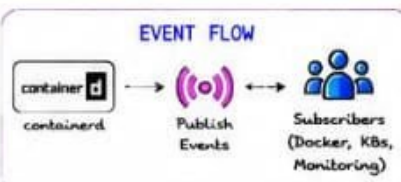
- Uses **snapshotters** to manage filesystem layers.
- Provides copy-on-write filesystem views.
- Snapshotters: overlayfs, btrfs, zfs, stargz, nydus, etc.
- Used for container writable layer.



6. EVENT HANDLING

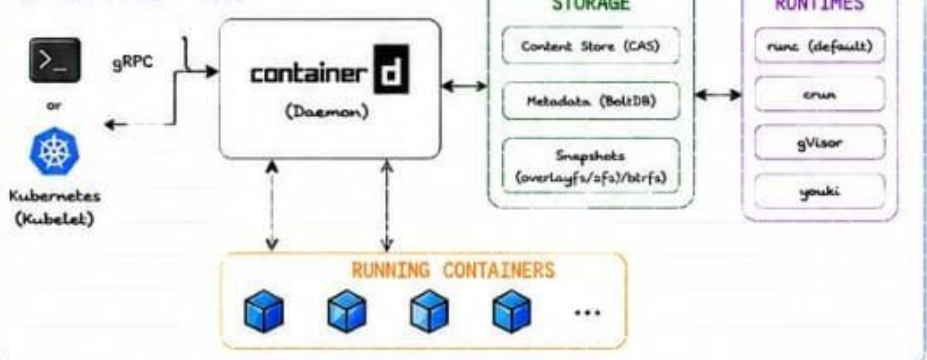


- containerd publishes events for all operations.
- Events are used by Docker, Kubernetes and monitoring systems.
- Example events: create, start, stop, delete, pull, task-exit.



7. PRODUCTION ARCHITECTURE

CONTROL PLANE / NODE



BENEFITS

- ✓ Lightweight & high performance
- ✓ Modular architecture
- ✓ Used by Docker, Kubernetes and cloud platforms
- ✓ OCI compliant
- ✓ Scalable and production ready

COMMON CONTAINERD COMMANDS (using ctr)

```
ctr images pull nginx:1.25
ctr images ls
ctr run -d --name web nginx:1.25
ctr tasks ls
ctr tasks kill <task-id>
ctr containers ls
ctr events
```

KEY TAKEAWAYS



- containerd is the engine that powers containers.
- It manages images, storage, tasks and events.
- It uses OCI runtimes to run containers.
- It is the default runtime for Kubernetes.
- Docker Engine communicates with it via gRPC.



containerd is the **heart** behind every modern container platform.
Master it, and you master the **engine**.





RUNC

★ The low-level OCI runtime used by containerd (and others) to create and run containers on Linux. ★

1. WHAT IS RUNC?

- runc is a lightweight CLI tool that implements the OCI Runtime Specification.
- It is responsible for spawning and running containers.
- It creates the container's process and manages its lifecycle.
- Written in Go, maintained by the OCI community.

KEY POINTS

- ✓ Implements OCI Runtime Spec
- ✓ Not a daemon
- ✓ Small, fast and secure
- ✓ Used by containerd, CRI-O, Podman, Docker (indirectly)
- ✓ Focuses only on runtime execution, not image management

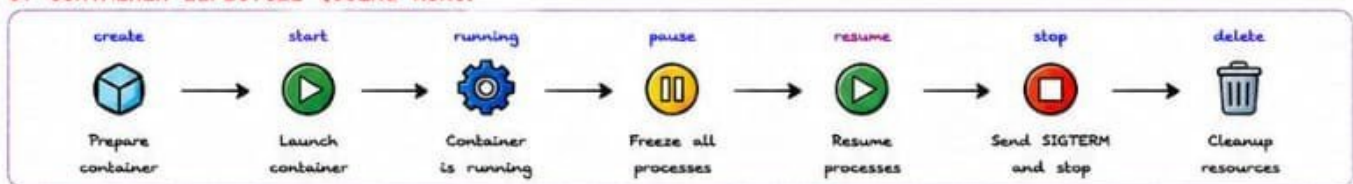
2. OCI RUNTIME

- runc follows the OCI Runtime Specification.
- It accepts a bundle (config.json + rootfs) and creates an isolated container process.
- It does NOT pull images, manage networks or storage.
- Its job = create + run the container process according to the spec.

OCI RUNTIME SPEC (inputs)

- config.json (runtime config)
 - rootfs (filesystem)
 - process (command to run)
- = OCI Bundle

3. CONTAINER LIFECYCLE (USING RUNC)



4. PROCESS CREATION (HOW RUNC STARTS A CONTAINER)

- runc receives the OCI bundle (config.json + rootfs path).
- runc creates namespaces (pid, net, mount, ipc, uts, user, cgroup) as defined in config.json.
- runc sets up cgroups for resource limits.
- runc mounts the rootfs (read-only layers + writable layer).
- runc sets up networking (via containerd/CNI).
- runc executes the container init process (usually /sbin/init or the app command).

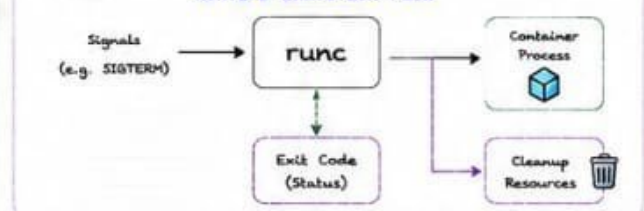
PROCESS CREATION FLOW

- Receive OCI Bundle (config.json + rootfs)
- Create Namespaces (Isolation)
- Setup Cgroups (Limits & Accounting)
- Setup Rootfs (Mount Filesystem)
- Setup Networking (NetNS & Interfaces)
- Exec Init Process (Start Container)

5. RUNTIME EXECUTION

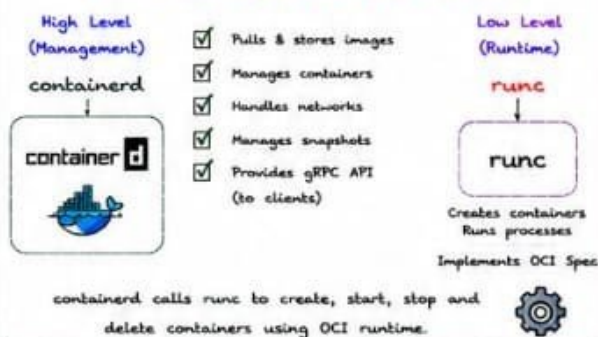
- Once the container process is started, runc monitors and manages it.
- It forwards signals (SIGTERM, SIGKILL, etc.).
- It waits for the process to exit and returns the exit code.
- It cleans up namespaces, cgroups and mounts when the container stops.

RUNTIME EXECUTION FLOW



6. RELATIONSHIP WITH CONTAINERD

HOW CONTAINERD AND RUNC WORK TOGETHER



7. RUNC COMMANDS (BASIC)

COMMAND	DESCRIPTION	EXAMPLE
runc list	List all containers	runc list
runc create <id>	Create container	runc create myctr
runc start <id>	Start container	runc start myctr
runc state <id>	Show container state	runc state myctr
runc exec <id> <cmd>	Exec command in container	runc exec myctr /bin/sh
runc pause <id>	Pause container	runc pause myctr
runc resume <id>	Resume container	runc resume myctr
runc stop <id>	Stop container	runc stop myctr
runc delete <id>	Delete container	runc delete myctr
runc kill <id> <sig>	Kill container process	runc kill myctr KILL

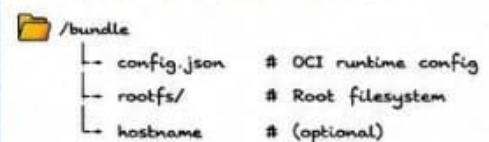
KEY TAKEAWAYS

- ✓ runc is the low-level OCI runtime.
- ✓ It creates and runs containers.
- ✓ It does NOT manage images or networks.
- ✓ It follows the OCI Runtime Specification.
- ✓ containerd (or CRI-O) uses runc for execution.

RUNC IS NOT...

- ✗ Not a daemon
- ✗ Not an image manager
- ✗ Not a network manager
- ✗ Not a high-level orchestrator

COMMON RUNC BUNDLE STRUCTURE



runc is the **engine** in the engine.
containerd is the **brain**, runc is the **muscle**.



Namespaces provide isolation by virtualizing system resources.
Each container gets its own view of the system.



A namespace wraps a global system resource in an abstraction so that it appears private to a set of processes.

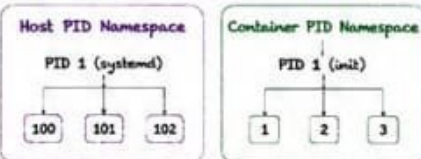


Used by: Docker, containerd, Kubernetes, runc, LXC
Implemented by: Linux Kernel

1. PID NAMESPACE



- Isolates process IDs.
- Processes in a namespace have their own PID 1 (init).
- Processes cannot see PIDs outside their namespace.

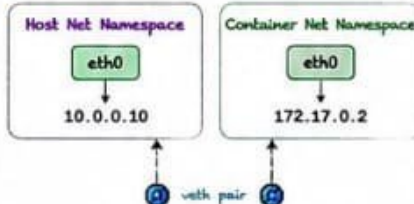


Example: ps inside container shows only processes in that container.

2. NETWORK NAMESPACE



- Provides isolated network stack, interfaces, routing tables, iptables.
- Each namespace has its own virtual network devices.

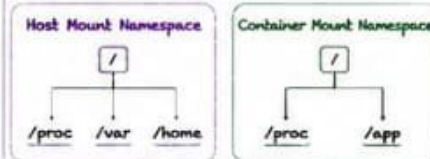


Example: ip addr inside container shows only its interfaces.

3. MOUNT NAMESPACE



- Isolates filesystem mounts.
- Containers can have their own view of the filesystem.
- Mounts/unmounts do not affect host or other containers.



Example: container can mount a volume at /data without affecting host.

4. IPC NAMESPACE



- Isolates IPC resources: System V IPC, POSIX message queues, shared memory, semaphores.
- Containers can't see IPC objects from other namespaces.



Example: shared memory in container is invisible to host.

5. UTS NAMESPACE



- Isolates hostname and NIS domain name.
- Each namespace can have its own hostname.



Example: hostname inside container can be different from host.

6. USER NAMESPACE



- Isolates user and group IDs.
- Allows containers to have their own UID/GID mapping.
- Enables rootless containers.

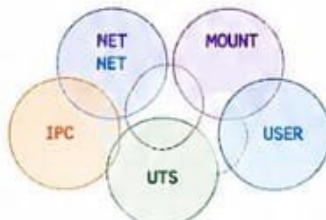


Example: root inside container can be mapped to non-root on host.

7. ISOLATION ARCHITECTURE

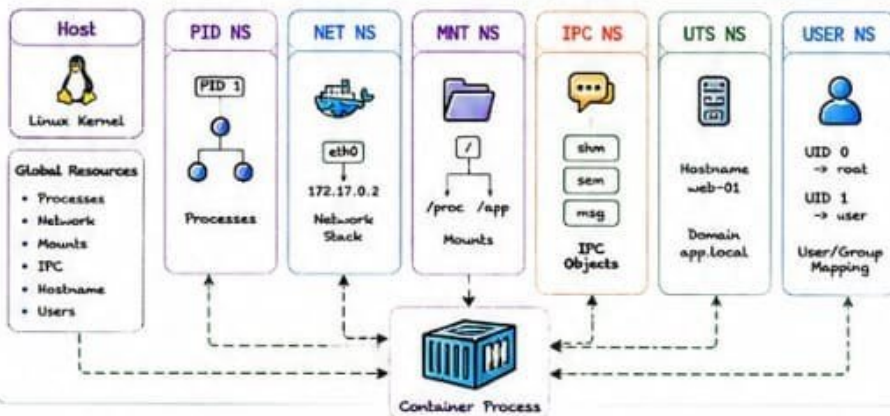


- Containers use a combination of namespaces to provide strong isolation.
- Once a process is in a namespace, it only sees the resources of that namespace.



Together = Container Isolation

NAMESPACE ISOLATION ARCHITECTURE



KEY TAKEAWAYS



- Namespaces isolate resources.
- Containers see only what they are allowed to see.
- Different namespaces work together to provide strong isolation.
- This is the foundation of containers on Linux.

SUMMARY TABLE

Namespace	Isolates	Example Resource
PID	Process IDs	ps, top
NET	Network Stack	Interfaces, IP, routes
MNT	Mount Points	Filesystem view
IPC	IPC Resources	Shared memory, semaphores
UTS	Hostname/Domain	hostname, domainname
USER	UID/GID Mapping	rootless containers

COMMON COMMANDS



- See namespaces of a process
lsns -p <pid>
- Enter a namespace (example PID)
nsenter -t <pid> -n bash # network ns
nsenter -t <pid> -m bash # mount ns
nsenter -t <pid> -u bash # UTS ns
- Create new namespaces (unshare)
unshare -fpimun bash



Namespaces are the walls that keep containers **isolated**.
Together, they make containers **secure** and **reliable**.





LINUX CGROUPS

☆ cgroups (control groups) limit, account and isolate resource usage of processes. ☆



cgroups let the kernel enforce resource limits and track usage for a group of processes.



Used by:

- Docker
- Kubernetes
- systemd
- Many other tools

1. CPU CONTROLLER



- Controls how much CPU time a group can use.
- Uses concepts: quotas and periods.
- Prevents noisy neighbor problems.
- Enforced by Completely Fair Scheduler (CFS).

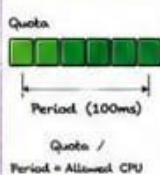
EXAMPLE

```
# Limit container to 1 CPU core (100%)
--cpus 1

# Limit to 50% of 2 cores
--cpus 1.0

# Low priority
--cpu-shares 128
```

CPU QUOTA MODEL



2. MEMORY CONTROLLER



- Limits the amount of memory a group can use.
- Prevents Out Of Memory (OOM) situations.
- Supports soft and hard limits.
- Can trigger OOM kill inside the group.

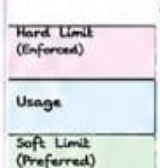
EXAMPLE

```
# Hard limit 512MB
--memory 512m

# Soft limit (preferred)
--memory-reservation 256m

# Swap limit
--memory-swap 1g
```

MEMORY LIMITS



3. BLOCK I/O (BLKIO)



- Controls read/write rates to block devices.
- Prevents one container from saturating disk I/O.
- Works per device (e.g., /dev/sda).

EXAMPLE

```
# Limit write to 10 MB/s
--device-write-bps /dev/sda:10mb

# Limit read to 20 MB/s
--device-read-bps /dev/sda:20mb

# Limit IOPS (read)
--device-read-iops /dev/sda:100
```

4. NETWORK LIMITS



- Limits network bandwidth for a group.
- Works on egress and ingress traffic.
- Measured in bytes per second.

EXAMPLE

```
# Limit egress to 5 MB/s
--network-rate 5242880

# Limit ingress to 2 MB/s
--network-rate 2097152

# Both directions
--network-rate 5242880:2097152
```

NETWORK LIMIT MODEL



5. RESOURCE ACCOUNTING



- cgroups keep track of actual resource usage.
- Helps in monitoring, alerting and billing.
- Usage files are available under cgroup fs.

COMMON ACCOUNTING METRICS

```
cpu.stat          # CPU usage stats
cpuacct.usage     # Total CPU time (ns)
memory.current    # Current memory usage
memory.max        # Memory limit
blkio.io_service_bytes # Bytes read/written
net_cls.classid   # Network class
pids.current      # Number of processes
pids.max          # PID limit
```

6. RESOURCE ISOLATION



- cgroups isolate resources between groups.
- Ensures one container cannot affect others.
- Provides predictable performance.

ISOLATION EXAMPLE

Container A	Container B	Container C
CPU: 1 core	CPU: 0.5 core	CPU: 0.5 core
MEM: 512MB	MEM: 256MB	MEM: 256MB
NET: 5MB/s	NET: 2MB/s	NET: 1MB/s
IOPS: 100	IOPS: 50	IOPS: 50

cgroups (Kernel Enforced Isolation)

7. PRODUCTION EXAMPLES



SCENARIO	CGROUP CONTROLLERS USED	GOAL	EXAMPLE (DOCKER/K8S)
Limit CPU hog container	cpu, cpuacct	Prevent container from stealing all CPU	docker run --cpus=0.5 myapp K8s: resources.limits.cpu: 500m
Prevent OOM on host	memory, memoryswap	Keep container within memory limit	docker run --memory=512m --memory-swap=1g K8s: resources.limits.memory: 512Mi
Protect disk performance	blkio	Avoid noisy neighbors on slow disks	docker run --device-read-bps /dev/sda:20mb K8s: emptyDir + QoS + Limits
Control network usage	net_cls, net_prio	Limit bandwidth per container/service	docker run --network-rate 5242880 K8s: CNI + NetworkPolicy + Limits
Track billing per tenant	cpuacct, memory, blkio	Measure usage for chargeback	cgroup metrics -> Prometheus/Grafana Tag by container label
Multi-tenant platform	All controllers	Strong isolation and fair sharing	Docker/K8s default cgroups Pod/Container level Limits

KEY TAKEAWAYS



- ✓ cgroups limit, account and isolate resources.
- ✓ Each controller handles a specific resource.
- ✓ They are the foundation of containers.
- ✓ Used by Docker, Kubernetes and systemd.

USEFUL COMMANDS



```
# Show cgroups mounted
mount | grep cgroup

# View current cgroups
cat /proc/self/cgroup

# Inspect a cgroup
cat /sys/fs/cgroup/<controller>/<group>/<file>

# Docker stats (uses cgroups metrics)
docker stats
```

CGROUP HIERARCHY (v2 example)

```
/sys/fs/cgroup
├── cpu
├── memory
├── io
├── pids
├── net_cls
└── system.slice
```

Each group has its own limits, usage and sub-groups.

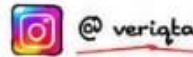


cgroups give you control, visibility and isolation.
The kernel enforces. You get performance and stability.





VERIQTA OVERLAYFS

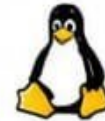


OverlayFS is a union filesystem for Linux.

★ It is the default storage driver (overlay2) used by Docker on most systems. ★



OverlayFS layers multiple directories into a single unified view and uses Copy-on-Write (CoW) for efficiency.



Used by:

- Docker (overlay2)
- Podman
- containerd
- Kubernetes

1. LOWER LAYER



- Read-only layers from image.
- Can be multiple lower directories (stacked).
- Never modified.

Example Path

`/var/lib/docker/overlay2/<id>/diff (RO)`

2. UPPER LAYER



- Writable layer for the container.
- Stores changes to files from lower layers.
- Only this layer is modified.

Example Path

`/var/lib/docker/overlay2/<id>-upper (RW)`

3. WORK DIRECTORY



- Used by OverlayFS for internal operations.
- Stores metadata about changes (CoW).
- Must be on the same filesystem as Upper.

Example Path

`/var/lib/docker/overlay2/<id>-work`

4. MERGED DIRECTORY



- The unified view that containers see.
- Combines Lower + Upper into a single directory.
- This is what the container mounts and uses.

Example Path

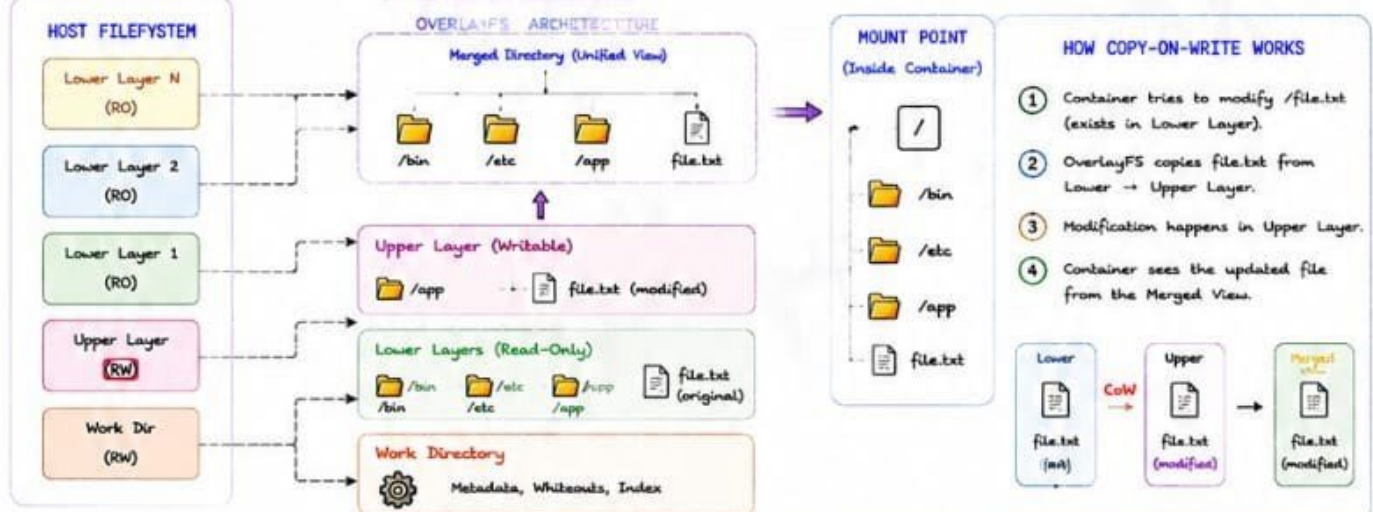
`/var/lib/docker/overlay2/<id>-merged (RO/RW view)`

5. COPY-ON-WRITE



- When a file from Lower is modified, it is copied to Upper first.
- Changes are made in Upper only.
- Lower layers remain untouched.

6. LAYER ARCHITECTURE



7. PERFORMANCE CONSIDERATIONS



FAST OPERATIONS ✓

- Reading from Lower Layers
- Container startup
- Layer sharing
- Disk space efficiency



SLOWER OPERATIONS ⚠

- First write to a file (CoW copy)
- Lots of small file writes
- Heavy metadata operations
- Deep layer stacks



BEST PRACTICES

- Keep images lean
- Minimize layers
- Group related commands in single RUN
- Use multi-stage builds



TIPS

- Store write-intensive data on volumes
- Avoid unnecessary file churn
- Use `..dockerignore`
- Keep containers stateless

8. EXAMPLE: FILE READ/WRITE FLOW

ACTION	WHERE IT HAPPENS	DETAILS
Read file (exists in lower)	Lower Layer	Direct read (fast)
Read file (exists in upper)	Upper Layer	Returns modified file
Modify file (exists in lower)	Upper Layer (CoW)	File copied to Upper, then modified
Create new file	Upper Layer	Created directly in Upper
Delete file	Upper + Whitout	Whitout marks deletion

9. OVERLAYFS STRUCTURE (overlay2 example)

`/var/lib/docker/overlay2/<id>/`

- └ diff/ → Lower layer content (RO)
- └ link → Short ID link
- └ Lower → List of lower layer links
- └ merged/ → Merged directory (mount)
- └ work/ → Work directory (internal)
- └ upper/ → Upper layer (writes)

10. KEY TAKEAWAYS ★

- ✓ OverlayFS uses multiple layers to build a unified view.
- ✓ Lower layers are read-only.
- ✓ All changes go to the Upper Layer.
- ✓ Copy-on-Write makes it efficient.
- ✓ It is the backbone of Docker storage.



OverlayFS makes containers **lightweight**, **fast** and **efficient** by sharing layers and using **Copy-on-Write**.



Docker images are built in layers.

★ Each instruction in a Dockerfile creates a new immutable layer. ★

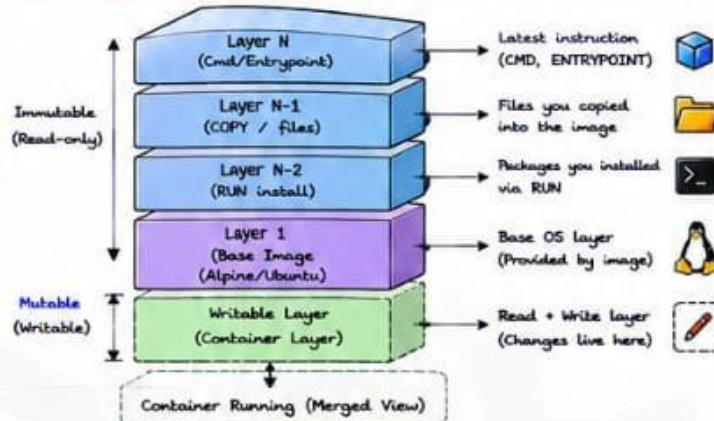


1. DOCKER IMAGE LAYER ARCHITECTURE



Think of an image as stacked pancakes.

- ✓ Each pancake = a layer
- ✓ You can't change old pancakes
- ✓ You add new ones on top



When Container Runs

Merged View (What you see inside)

```

/
├── bin
├── etc
├── usr
└── app
    
```

- ✓ All layers stacked
- ✓ Unified filesystem
- ✓ Looks like a normal OS
- ✓ But built from layers

2. READ-ONLY VS WRITABLE LAYER

Read-Only Layers (Immutable)

- ✓ Created at build time
- ✓ Cannot be changed
- ✓ Shared across containers
- ✓ Cached by Docker
- ✓ Persist in image



Writable Layer (Container Layer)

- ✓ Created at run time
- ✓ Can be changed
- ✓ Specific to container
- ✓ Not part of image
- ✓ Lost when container deleted

3. HOW LAYERS ARE CREATED (Dockerfile Example)

Dockerfile

```

FROM ubuntu:22.04
RUN apt update
RUN apt install -y nginx
COPY . /app
WORKDIR /app
CMD ["nginx", "-g", "daemon off;"]
    
```

Resulting Layers



4. LAYER CACHE (Docker's Superpower)

- Docker caches layers by the instruction + its content.
- If nothing changes, Docker reuses the layer from cache.
- This makes builds extremely fast.

First Build (No Cache)

- ✗ Step 1 → Build
- ✗ Step 2 → Build
- ✗ Step 3 → Build
- ✗ Step 4 → Build

Second Build (With Cache)

- ✓ Step 1 → Cached
- ✓ Step 2 → Cached
- ✗ Step 3 → Rebuild (changed)
- ✗ Step 4 → Build

Result: 70-90% faster

5. LAYER REUSE (Across Images)



6. IMAGE OPTIMIZATION TIPS

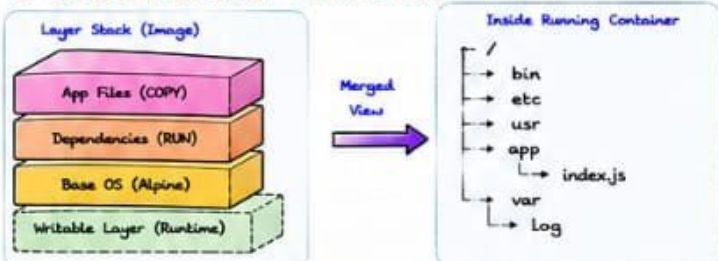
- ✓ Use small base images (alpine, distroless, slim)
- ✓ Combine RUN commands to reduce layers
- ✓ Remove caches after install (apt clean, rm -rf /var/lib/apt/lists/*)
- ✓ Use .dockerignore to exclude unnecessary files
- ✓ Multi-stage builds for smaller final images
- ✓ Keep layers ordered wisely (Less changing layers at bottom)

7. LAYER SIZE IMPACT

Layer	Instruction	Size	Cumulative Size
Base OS (Alpine)	FROM alpine:3.19	7 MB	7 MB
Install Nginx	RUN apt add nginx	12 MB	19 MB
Copy App	COPY . /app	5 MB	24 MB
Add Config	COPY nginx.conf /etc/nginx/	200 KB	24.2 MB
Final CMD	CMD ["nginx", "-g", "daemon off;"]	50 KB	24.25 MB

Cumulative size is what matters.
Each new layer adds to the total image size.

8. VISUAL: LAYER STACK + MERGED VIEW



9. MULTI-STAGE BUILD (Smaller Images)



10. KEY TAKEAWAYS

- ✓ Images are built in layers.
- ✓ Layers are immutable and read-only.
- ✓ Writable layer is created at runtime.
- ✓ Docker caches layers to speed up builds.
- ✓ Layers are reused across images.
- ✓ Optimize your Dockerfile for small, fast, efficient images.

COMMON LAYER COMMANDS

- `docker history <image>` # See layers
- `docker image ls` # List images
- `docker image inspect <image>` # Deep info
- `docker image prune -a` # Remove unused images
- `docker system df` # See disk usage



Layers make Docker fast, efficient, and scalable.
Understand layers, and you understand Docker.





COPY-ON-WRITE (CoW)

Copy-on-Write is the mechanism that makes Docker layers efficient.

★ Files from lower layers are shared and only copied when modified. ★



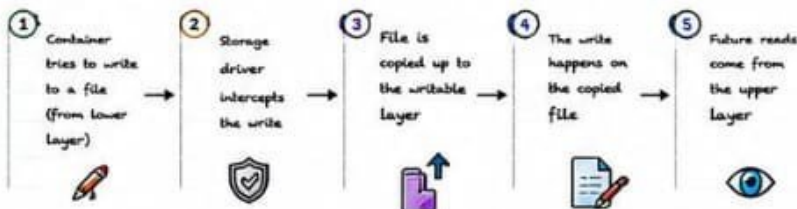
CoW means a file is not copied until it is changed.
This saves storage and improves efficiency.



Used by:

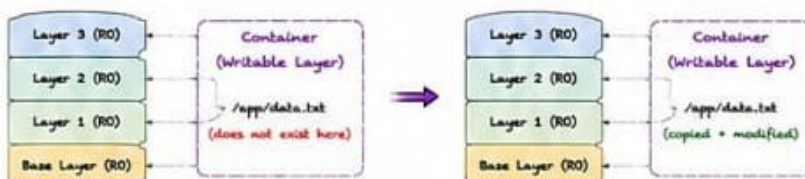
- overlay2
- btrfs
- zfs
- fuse-overlayfs

1. COPY-ON-WRITE PROCESS



BEFORE WRITE

AFTER WRITE

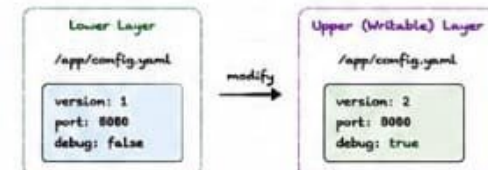


Only the changed file exists in the writable layer.

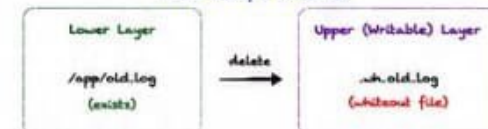
2. FILE MODIFICATIONS

When a file is modified:

- Original file in lower layer remains unchanged.
- A copy is created in the upper (writable) layer.
- Only the copy is modified.
- Deleting a file creates a "whiteout" file.



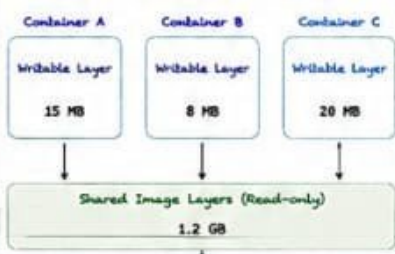
Delete Example (Whiteout)



3. STORAGE EFFICIENCY

- Files are shared between containers.
- Only changes consume additional space.
- Greatly reduces disk usage.
- Ideal for large base images.

EXAMPLE: 3 CONTAINERS FROM SAME IMAGE



Total Space Used = 1.2 GB + 15 MB + 8 MB + 20 MB
(Not 3 x 1.2 GB!)

4. PERFORMANCE IMPACT

- First write to a file = slower (File copy happens).
- Subsequent writes = faster (already in upper layer).
- Many small writes = more CoW operations.
- Can cause higher I/O and metadata overhead.
- Deep layer stacks = more lookups.

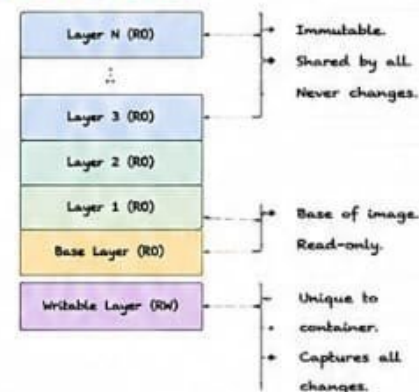
IMPACT SUMMARY

Scenario	Impact
First write (CoW copy)	Slower
Many small writes	Slower
Read operations	Fast (from lower)
Few writes	Minimal impact
Optimal layer design	Better performance

Tips:

- ✓ Keep layers few and meaningful
- ✓ Avoid writing to many unique files at runtime

5. LAYER BEHAVIOR



Whiteout files in RW layer
hide deleted files from
lower layers.

6. PRODUCTION EXAMPLES

EXAMPLE 1: LOG FILE GROWTH

- App writes logs to `/var/log/app.log`
- First write copies the file to writable layer.
- File grows here, not in image.
- Logs disappear when container is removed (unless mounted).

Best Practice:
Mount logs to a volume.



EXAMPLE 2: DATABASE FILES

- DB writes to `/var/lib/mysql`
- CoW causes heavy I/O and poor performance.
- Lots of small file writes increase CoW overhead.

Best Practice:
Use volumes for DB data.



EXAMPLE 3: CONFIG CHANGES

- Modify `/etc/nginx/nginx.conf`
- File copied to writable layer.
- Restart not needed, but change is container-specific.
- New container = original config.

Best Practice:
Use bind mounts for configs.



EXAMPLE 4: TEMP FILES

- App creates temp files in `/tmp`
- Files live in writable layer.
- Many temp files = more disk usage and cleanup cost.

Best Practice:
Use tmpfs mount for `/tmp`.

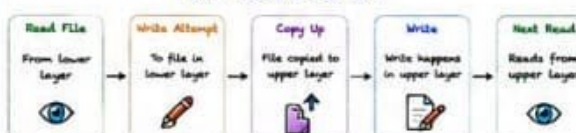


KEY TAKEAWAYS



- ✓ CoW copies files only when needed.
- ✓ Lower layers stay unchanged.
- ✓ Saves storage and speeds up deployments.
- ✓ First write is slow, later writes are fast.
- ✓ Use volumes for write-heavy data.
- ✓ Design images for minimal writes.

CoW VISUAL SUMMARY



Copy happens once. Benefit forever.

QUICK COMMANDS



- See layer sizes
`docker image inspect <image> --format '{{.Size}}'`
- See layer details
`docker history <image>`
- See container layer usage
`docker system df -v`
- Clean up unused data
`docker system prune -af --volumes`



Use Copy-on-Write wisely.
Volumes for data, layers for code.
Efficient storage. Faster containers. Happy production.



DOCKER STORAGE DRIVERS



Storage drivers determine how Docker stores **image layers** and container data on the host filesystem.



A storage driver is a pluggable backend that manages:

- ✓ How Docker stores image layers
- ✓ How containers write data
- ✓ How layers are combined and mounted



Used by:

- Docker Engine
- Podman
- Kubernetes (via containerd/Docker)
- Build tools (BuildKit, Kaniko, etc.)

1. OVERLAY2



- Default on most Linux distributions.
- Based on OverlayFS (kernel feature).
- Fast, stable and widely supported.

Best For:

- General purpose workloads
- Production environments
- Kubernetes nodes

2. FUSE-OVERLAYS



- Userspace implementation of OverlayFS.
- Works in unprivileged environments (rootless).
- Slightly slower than overlay2.

Best For:

- Rootless Docker/Podman
- CI/CD runners
- Unprivileged users

3. BTRFS



- Copy-on-write filesystem.
- Built-in snapshot support.
- Subvolume based.
- Strong data integrity features.

Best For:

- Systems needing snapshots
- Backup & rollback
- Dev/Test environments

4. ZFS



- Enterprise-grade filesystem.
- Advanced features: snapshots, compression, deduplication, encryption.
- Requires more resources.

Best For:

- Large-scale systems
- Data integrity critical apps
- Enterprise workloads

5. DEVICEMAPPER



- Uses block device (LVM) for storage.
- Older but still supported.
- Thin provisioning supported.

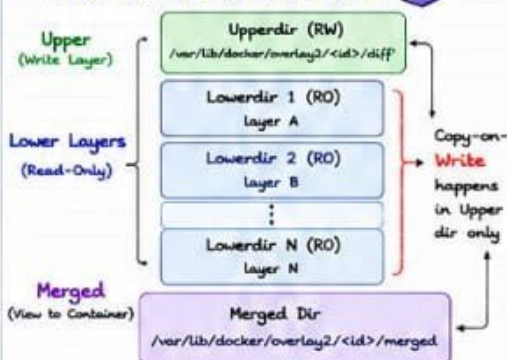
Best For:

- Legacy systems
- LVM-based environments
- Backward compatibility

6. STORAGE DRIVERS COMPARISON

Driver	Kernel Required	Performance	Snapshot Support	Space Efficiency	Rootless Support	Data Integrity	Complexity	Best Use Case
overlay2	Yes (4.0+)	★★★★★	Limited (manual)	High	✓	Good	Low	General purpose, Kubernetes, Production
fuse-overlays	No (FUSE)	★★★★☆	Limited	Medium	✓✓✓ (Excellent)	Good	Medium	Rootless, CI/CD, Userspace
btrfs	Yes (3.10+)	★★★★☆	✓✓✓ (Built-in)	High	✓	Excellent	Medium	Snapshots, Dev/Test, Backups
zfs	Yes (ZFS Module)	★★★★☆	✓✓✓ (Advanced)	Very High	✗ (Not native)	Excellent	High	Enterprise, Data critical, Large scale
devicemapper	Yes (LVM)	★★★☆☆	✓ (Thin Pools)	Medium	✗ (Limited)	Good	High	Legacy, LVM environments, Compatibility

7. OVERLAY2 LAYER ARCHITECTURE



8. PERFORMANCE (Relative)



Note: Performance depends on hardware (disk type, I/O, RAM) and workload.

9. BEST PRACTICES

- ✓ Use overlay2 for most Linux systems.
- ✓ Use fuse-overlays for rootless environments.
- ✓ Use btrfs or ZFS when you need snapshots, rollback or strong data integrity.
- ✓ Use devicemapper only when required for legacy compatibility.
- ✓ Keep `/var/lib/docker` on fast SSD storage.
- ✓ Monitor disk usage and clean up unused images/containers.
- ✓ Use `.dockerignore` and multi-stage builds to reduce image size.



10. DRIVER DEEP DIVE (QUICK NOTES)

- **overlay2** → Kernel overlays + ext4/xfs
✓ Fast ✓ Stable ✓ Default
- **fuse-overlays** → Userspace overlay via FUSE
✓ Rootless ✓ Portable ✗ Slightly slower
- **btrfs** → CoW filesystem with subvolumes
✓ Snapshots ✓ Compression ✗ Checksums
- **zfs** → Enterprise filesystem
✓ Snapshots ✓ Compression ✓ Dedup ✗ Heavy on RAM
- **devicemapper** → LVM thin provisioning
✓ Thin pools ✓ Backward compatible ✗ Older



11. COMMON LOCATIONS

Docker images & layers:
`/var/lib/docker`

Overlay2 layers:
`/var/lib/docker/overlay2/`

Docker data root (can change):
`/etc/docker/daemon.json`

Check active driver:
`docker info | grep "Storage Driver"`

12. QUICK COMMANDS

- ✓ Check storage driver:
`docker info | grep "Storage Driver"`
- ✓ List Docker system usage:
`docker system df -v`
- ✓ Inspect a container layer:
`docker inspect <container_id>`
- ✓ Clean unused data:
`docker system prune -a --volumes`
- ✓ Change data root (in `daemon.json`):
"data-root": `"/mnt/fast-disk/docker"`



Choose the right **storage driver**. Store **efficiently**. Run **faster**.
Reliable storage = Reliable containers.





DOCKER NETWORKING INTERNALS

★ Docker uses Linux networking primitives to provide isolated, efficient and flexible container networking. ★



At its core, Docker networking is built on namespaces, virtual ethernet devices and Linux bridges.



Used by:

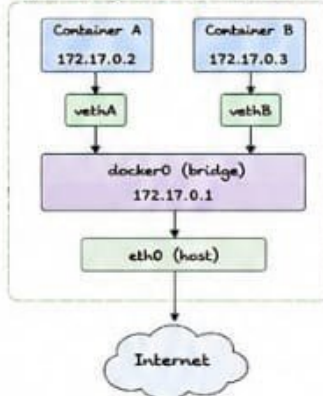
- Docker Engine
- containerd
- Kubernetes (via CNI)
- Microservices

1. LINUX BRIDGE

- Docker creates a Linux bridge (docker0 by default).
- Acts like a virtual switch.
- Connects containers on the same host.
- Provides NAT to reach external networks.

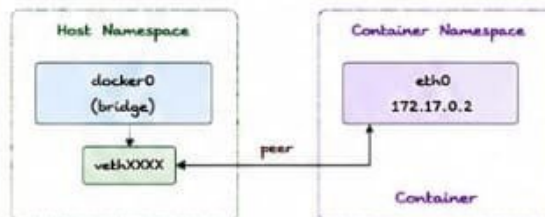
docker0 (default)	
Subnet	172.17.0.0/16
Gateway	172.17.0.1
Driver	bridge
IP Masquerade	Enabled (NAT)

Default Bridge Network Architecture



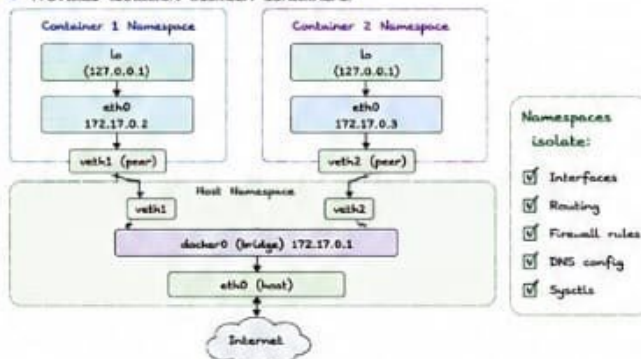
2. VETH PAIRS

- veth (virtual ethernet) creates a pair of connected interfaces.
- One end in the container namespace.
- One end attached to the bridge on the host.
- Every container gets its own veth pair.



3. NETWORK NAMESPACES

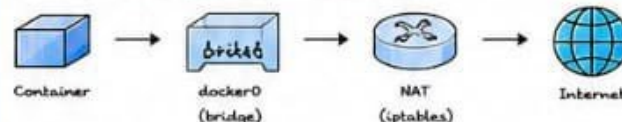
- Each container gets its own network namespace.
- It has its own network stack (interfaces, routing table, iptables, etc).
- Provides isolation between containers.



Namespaces isolate:

- ✓ Interfaces
- ✓ Routing
- ✓ Firewall rules
- ✓ DNS config
- ✓ Sysctls

4. PACKET FLOW (CONTAINER TO INTERNET)



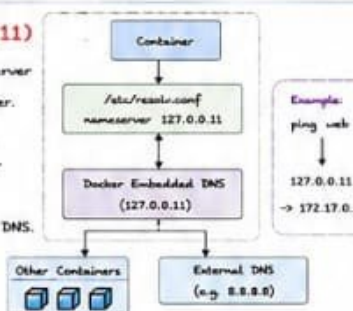
1. Container sends packet via its eth0.
2. Packet goes to its peer veth on the host.
3. Host veth forwards to docker0 bridge.
4. If destination is external, iptables (MASQUERADE) applies NAT.
5. Packet goes out via host's eth0 to the Internet.

Common iptables rules (simplified)

PREROUTING	DNAT (for published ports)
FORWARD	Allow container traffic
POSTROUTING	MASQUERADE (SNAT to host IP)

5. EMBEDDED DNS (127.0.0.11)

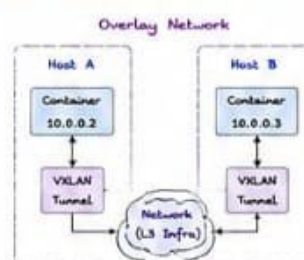
- Docker runs an embedded DNS server at 127.0.0.11 inside each container.
- Resolves container names to IPs.
- Supports service discovery on user defined networks.
- Forwards unknown queries to host DNS.



Example:
ping web
127.0.0.11
→ 172.17.0.5

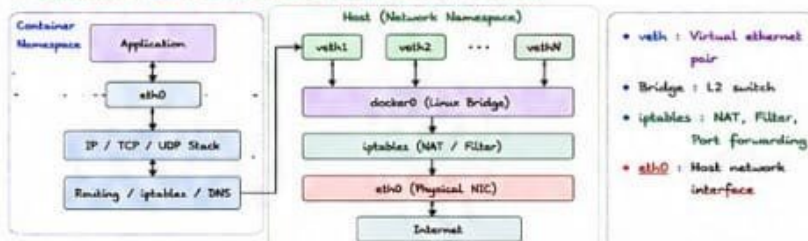
6. OVERLAY NETWORKING

- Used for multi-host communication.
- Built on VXLAN (UDP 4789).
- Creates a virtual Layer 2 network across Docker hosts.
- Used by Swarm and Kubernetes overlay drivers.



Same subnet (10.0.0.0/24) across hosts

7. INTERNAL ARCHITECTURE (DOCKER NETWORK STACK)



- veth : Virtual ethernet pair
- Bridge : L2 switch
- iptables : NAT, Filter, Port forwarding
- eth0 : Host network interface

8. COMMON DOCKER NETWORK DRIVERS

Driver	Description
Bridge	Default. For containers on the same host.
host	Container shares the host network stack.
none	No networking.
overlay	Multi-host networking (Swarm, K8s).
macvlan	Assign MAC address to container.
ipvlan	L3 mode, lightweight macvlan alternative.

BEST PRACTICES

- ✓ Use user-defined bridge networks.
- ✓ Avoid --link (deprecated).
- ✓ Use overlay for multi-host setups.
- ✓ Secure published ports.
- ✓ Monitor network with tools (e.g. nicolaka/netshoot).

KEY TAKEAWAYS



- Docker networking = namespaces + veth + bridge.
- Embedded DNS (127.0.0.11) enables service discovery.
- iptables provides NAT and port publishing.
- Overlay networks enable containers across hosts.

USEFUL COMMANDS

```

docker network ls
docker network inspect <net>
ip a
brctl show
iptables -t nat -L -n -v
nsenter -t <pid> -n bash
# list networks
# inspect network
# show interfaces
# list bridges
# view NAT rules
# enter net namespace
  
```



Docker networking is powerful because it leverages Linux primitives. Understand the building blocks, master the system.



Buildx is the extended build capability for Docker.

★ It enables multi-platform builds, advanced caching and remote builders. ★



Buildx + BuildKit = Fast, portable and powerful builds for modern container delivery.



Used by:

- Developers
- Platform Engineers
- DevOps Teams
- CI/CD Pipelines

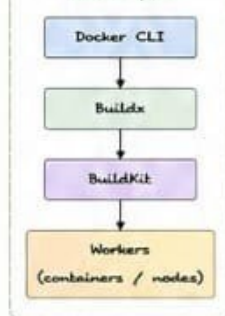
1. BUILDX OVERVIEW

- Buildx is a Docker CLI plugin.
- Extends docker build with BuildKit features.
- Supports multi-platform images.
- Advanced caching and exporters.
- Works with local and remote builders.

Default command

```
docker buildx build [OPTIONS] PATH | URL | -
```

How it fits



2. BUILDER INSTANCES

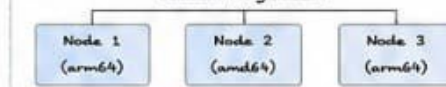
- Buildx uses builder instances to perform builds.
- Each builder can have multiple nodes.
- Builders can be local, remote or cloud based.

Manage builders

```
docker buildx ls
docker buildx create --name mybuilder --use
docker buildx inspect mybuilder
docker buildx rm mybuilder
```

```
# List builders
# Create and use
# Inspect builder
# Remove builder
```

Builder: mybuilder



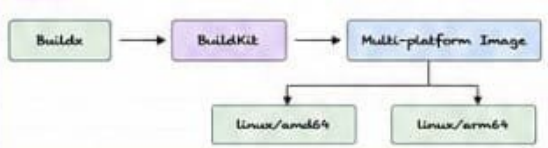
3. MULTI-PLATFORM BUILDS

- Buildx once, run anywhere.
- Buildx can create images for multiple architectures in a single command.
- Uses QEMU for emulation (when needed).

Example:

```
docker buildx build --platform linux/amd64,linux/arm64 \
-t myapp:latest --push
```

Result:



4. CACHE EXPORT / IMPORT

- Cache layers can be exported to a registry, local directory or inline.
- Speeds up builds across CI/CD runs and machines.

Export cache

```
docker buildx build --cache-to type=registry,ref=myrepo/cache:build --push
```

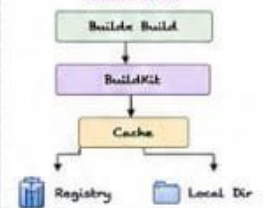
Import cache

```
docker buildx build --cache-from type=registry,ref=myrepo/cache:build
```

Supported Cache Types

Type	Description
inline	Embed cache in the image (default)
registry	Store cache in an OCI registry
local	Store cache in local directory
gha	GitHub Actions cache backend
s3	Store cache in Amazon S3 (via plugin)

Cache Flow



5. CROSS COMPILE

- Buildx + BuildKit supports cross-compilation.
- Provide build args, platforms and use multi-stage builds.
- Works great with Go, Rust, Node.js, .NET, etc.

Example (Go app for multiple platforms):

```
# syntax=docker/dockerfile:1.6
FROM --platform=$BUILDPLATFORM golang:1.22 AS build
ARG TARGETOS
ARG TARGETARCH
WORKDIR /src
COPY . .
RUN GOOS=$TARGETOS GOARCH=$TARGETARCH go build -o /out/app
FROM scratch
COPY --from=build /out/app /app
ENTRYPOINT ["/app"]
```

Build Args

```
BUILDPLATFORM = where build runs
TARGETPLATFORM = final target
(e.g. linux/arm64)
```

Platforms

- linux/amd64
- linux/arm64
- linux/arm/v7
- windows/amd64
- darwin/amd64
- darwin/arm64

6. PRODUCTION WORKFLOW



1. Setup Builder

Create a dedicated builder with remote nodes (amd64, arm64).

2. Build

Build multi-platform image with caching enabled.

3. Test

Run tests in ephemeral containers. Validate.

4. Push

Push multi-platform image and cache to registry.

5. Deploy

Deploy in Kubernetes / ECS / Nomad / Anywhere.

Example CI/CD Command

```
docker buildx build --platform linux/amd64,linux/arm64 \
--cache-from type=registry,ref=myrepo/cache:build \
--cache-to type=registry,ref=myrepo/cache:build,mode=max \
--tag myrepo/myapp:1.0.0 --push
```



7. USEFUL BUILDX COMMANDS

```
docker buildx version # Show version
docker buildx ls      # List builders
docker buildx create --name b --use # Create & use
docker buildx inspect --bootstrap # Inspect & bootstrap
docker buildx build [opts] # Build image
docker buildx bake     # Build with HCL
docker buildx du       # Disk usage (cache)
docker buildx prune    # Clean cache
docker buildx stop b   # Stop builder
docker buildx rm b     # Remove builder
```



8. BEST PRACTICES

- ✓ Use dedicated builders for CI/CD.
- ✓ Enable caching (registry or gha).
- ✓ Push multi-platform images.
- ✓ Pin base images by digest.
- ✓ Use .dockerignore to reduce context size.
- ✓ Use multi-stage builds.
- ✓ Monitor builder disk usage.
- ✓ Keep QEMU binfmt updated on hosts.



9. KEY TAKEAWAYS

- Buildx extends Docker build with BuildKit power.
- Supports multi-platform, caching and remote builds.
- Builders = one orchestrator, many nodes.
- Cache export/import drastically speeds up builds.
- Essential for modern, scalable container delivery.

Build once.
Ship anywhere.
Run everywhere.



Buildx + BuildKit = Faster builds, smaller images, multi-platform everywhere!
Use it in development. Use it in CI/CD. Use it in production.





MULTI-PLATFORM IMAGES

Build once. Run anywhere.

★ Deliver the right image for every architecture. ★



Multi-platform images use a manifest list to point to architecture-specific images. Docker pulls the right one automatically.



Used by:

- Developers
- Platform Engineers
- DevOps Teams
- Release Engineers

1. WHAT IS A MANIFEST LIST?

- A manifest list is a JSON object that references multiple image manifests.
- Each manifest points to an image for a specific platform.
- Docker Engine selects the best match for your machine.
- Also called an "image index".

Analogy:

A manifest list is like a menu. The menu (manifest list) points to dishes (images) for each platform.

Manifest List (Index)

Tag: myapp:1.0.0

Manifest (linux/amd64)

→ Image A

Manifest (linux/arm64)

→ Image B

Manifest (linux/arm/v7)

→ Image C

...

3. CROSS-PLATFORM BUILDS

- Use Docker Buildx + BuildKit to build for multiple platforms.
- QEMU emulation is used automatically (when needed).
- Build once → Push → Everyone pulls the right image.

Build Command Example

```
docker buildx build \
  --platform linux/amd64,linux/arm64 \
  -t myapp:1.0.0 \
  --push
```

Result

Image (amd64)

Image (arm64)

2. SUPPORTED ARCHITECTURES (COMMON)

ARCHITECTURE	OS	DESCRIPTION
AMD64	linux/amd64	64-bit x86 (Intel/AMD)
ARM64	linux/arm64	64-bit ARM (Graviton, Apple M1/M2, etc.)
ARMv7	linux/arm/v7	32-bit ARM
ARMv6	linux/arm/v6	Older 32-bit ARM
PPC64LE	linux/ppc64le	PowerPC Little Endian
S390x	linux/s390x	IBM Z mainframe
RISC-V 64	linux/riscv64	RISC-V 64-bit

Tip: Start with linux/amd64 and linux/arm64. They cover most modern workloads.



4. HOW IT WORKS (PULL FLOW)



1. Client pulls myapp:1.0.0
2. Docker Engine fetches the manifest list from registry.
3. Engine checks local platform (e.g., linux/arm64).
4. Selects the matching manifest (e.g., arm64 image).
5. Pulls and runs the correct image.

Verify Locally

```
docker buildx imagetools inspect myapp:1.0.0
```

5. IMAGE PUBLISHING

- When you push a multi-platform build, all platform images + manifest list are pushed together.
- Registry stores:
 - ☒ Platform-specific images
 - ☒ Manifest List (index)
- Consumers just pull the tag.

Registry (e.g., Docker Hub, ECR)

myapp:1.0.0 (Manifest List)

AMD64

Image

ARM64

Image

ARMv7

Image

6. PRODUCTION EXAMPLE



7. END-TO-END EXAMPLE (FROM CODE TO PRODUCTION)

1. Write Dockerfile

```
Dockerfile
FROM alpine
COPY ./app
CMD ["./app"]
```

2. Create Builder

```
docker buildx create \
  --name multi \
  --use
```

3. Build Multi-Platform

```
docker buildx build \
  --platform linux/amd64,linux/arm64 \
  -t myorg/myapp:1.0.0 \
  --push
```

4. Pushed to Registry

Registry stores:

- Manifest list
- amd64 image
- arm64 image

5. Deployed Anywhere

Each platform pulls the right image.

8. MANIFEST LIST INSPECT EXAMPLE

```
docker buildx imagetools inspect myapp:1.0.0
```

Name: myapp:1.0.0

MediaType: application/vnd.oci.image.index.v1+json

Manifests:

PLATFORM	DIGEST	SIZE
linux/amd64	sha256:aaa111...	18.2 MB
linux/arm64	sha256:bbb222...	17.9 MB
linux/arm/v7	sha256:ccc333...	16.1 MB

9. BEST PRACTICES

- ☒ Build for at least linux/amd64 and linux/arm64.
- ☒ Use Buildx + BuildKit.
- ☒ Test images on target architectures.
- ☒ Keep base images small and multi-arch.
- ☒ Pin base image versions.
- ☒ Use --cache-to and --cache-from for speed.
- ☒ Automate builds in CI/CD.
- ☒ Monitor image size per architecture.



10. USEFUL COMMANDS

- ```
Create & use builder
docker buildx create --use

List builders
docker buildx ls

Inspect builder
docker buildx inspect

Build & push multi-platform
docker buildx build --platform linux/amd64,linux/arm64 \
 -t myorg/myapp:1.0.0 --push

Inspect manifest list
docker buildx imagetools inspect myapp:1.0.0
```



Build once. Publish once. Run everywhere.  
Multi-platform images = Happy users + Portable deliveries.







## DOCKER PERFORMANCE INTERNALS

★ Understand what happens under the hood to make containers fast, efficient and production-ready. ★



Performance in Docker depends on smart resource usage, efficient storage, minimal overhead and proper tuning.

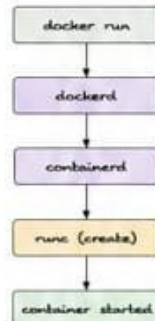


Used by:

- DevOps Engineers
- SREs
- Platform Engineers
- Performance Engineers

## 1. CONTAINER STARTUP PROCESS

1. Docker CLI sends request to daemon (dockerd)
2. Image layers are checked (cache hit/miss)
3. Container is created (namespaces + cgroups)
4. Root filesystem (OverlayFS) mounted
5. Network namespace + veth pair created
6. Process is started (ENTRYPOINT / CMD)
7. Container is now running



Key Takeaway:

Most time is spent in image pull, layer extraction, filesystem mount and network setup.

## 2. MEMORY MANAGEMENT

- Containers share the host kernel but have isolated memory.
- Cgroups limit and account memory usage.
- Memory is allocated on demand (lazy allocation).
- Uses copy-on-write to avoid unnecessary duplication.
- Swap is controlled by memory cgroup.



Tips:

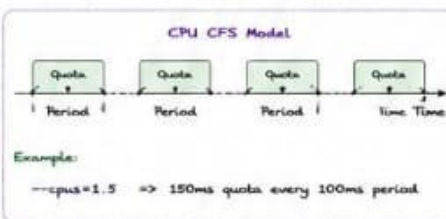
- ✓ Set memory limits (`--memory`) to prevent OOM.
- ✓ Avoid swap in production (use `--memory-swap=MEM`).
- ✓ Monitor usage: `docker stats` & `cgroup memory`.

Common Issues

- OOM kills
- Memory leaks
- Too much cache
- Swap thrashing

## 3. CPU SCHEDULING

- Docker containers use the host kernel scheduler (CFS).
- Cgroups CPU controller limits and shares CPU time.
- By default, containers have no CPU limit.
- Use `--cpus`, `--cpu-quota` and `--cpu-period` for control.



Tips:

- ✓ Set CPU limits for critical services.
- ✓ Use CPU requests/limits in Kubernetes.

## 4. DISK PERFORMANCE

- Storage driver and filesystem heavily impact performance.
- OverlayFS incurs copy-on-write overhead.
- Writing to container layer is slower than volumes.
- I/O can be limited by host disk, driver and queue depth.



Best Practices

- ✓ Use volumes for write-heavy workloads.
- ✓ Prefer SSD/NVMe for `/var/lib/docker`.
- ✓ Use overlay2 (better than devicemapper).
- ✓ Avoid small random writes on container layer.

Monitor

- `iostat -x 1`
- `df -h`
- `docker system df`
- `iostat / pidstat -d`

## 5. LAYER OPTIMIZATION

- Smaller images = faster pull, start and less disk usage.
- Fewer layers = less overhead.
- Order Dockerfile steps from least to most frequently changed.
- Clean up caches and unnecessary files.

Bad (cache busting) ❌

```
FROM ubuntu:22.04
COPY . /app
RUN apt-get update && apt-get install -y \
 curl git build-essential
RUN pip install --requirements.txt
RUN echo "version = v1" >> /app/version.txt
```

Good (cache friendly) ✅

```
FROM ubuntu:22.04
RUN apt-get update && apt-get install -y \
 curl git build-essential
COPY requirements.txt /app/
RUN pip install -- /app/requirements.txt
COPY . /app
CMD ["python", "app.py"]
```

Remember: Change at the bottom, not the top. 🎯

## 6. RUNTIME TUNING

- Small tuning can improve stability and performance.
- Tune container runtime, resources and kernel settings.

| AREA           | TUNING                       | EXAMPLE                                  |
|----------------|------------------------------|------------------------------------------|
| Resources      | Set proper limits            | <code>--cpus=1.5 --memory=2g</code>      |
| Logging        | Use json-file with rotation  | <code>--log-opt max-size=10m</code>      |
| Storage        | Use volumes for heavy writes | <code>-v /data:/data</code>              |
| Networking     | Use host mode (when needed)  | <code>--network host</code>              |
| Ulimits        | Increase file descriptors    | <code>--ulimit nofile=65536:65536</code> |
| Healthchecks   | Add healthcheck              | <code>HEALTHCHECK CMD curl -f ...</code> |
| Restart policy | Auto recover                 | <code>--restart unless-stopped</code>    |
| Swappiness     | Disable swap usage           | <code>--memory-swappiness=0</code>       |
| OOM Score      | Protect critical containers  | <code>--oom-score-adj -500</code>        |

## 7. PERFORMANCE TOOLS

- 🔧 `docker stats` Real-time CPU, MEM, NET, I/O
- 🔧 `docker top <id>` View processes inside container
- 🔧 `nsenter` Debug inside namespaces
- 🔧 `strace -p <pid>` Trace syscalls
- 🔧 `perf` CPU profiling
- 🔧 `iostat / pidstat` I/O and system metrics
- 🔧 `vmstat / sar` System performance summary
- 🔧 `cAdvisor` Container metrics exporter
- 🔧 `Prometheus + Grafana` Monitoring & dashboards

What to Monitor

- ✓ CPU usage & throttling
- ✓ Memory usage & limits
- ✓ Disk I/O & latency
- ✓ Network throughput
- ✓ Restarts & OOMs
- ✓ Container start time

Rule of Thumb

Measure → Identify → Tune → Verify → Repeat

## 8. COMMON PERFORMANCE ISSUES &amp; FIXES

| ISSUE                | CAUSE                      | HOW TO FIX                                   |
|----------------------|----------------------------|----------------------------------------------|
| Slow container start | Large image / many layers  | Optimize Dockerfile, use smaller base        |
| High CPU usage       | No CPU limit / busy loop   | Set <code>--cpus</code> , debug app, profile |
| Memory OOM           | No memory limit / leak     | Set <code>--memory</code> , fix memory leak  |
| Slow disk writes     | Writing to container layer | Use volumes, better storage                  |
| High I/O wait        | Disk bottleneck            | Use SSD/NVMe, tune I/O                       |
| Network latency      | Overlay network overhead   | Use host network / tune MTU                  |
| Log file growth      | No log rotation            | Configure log-driver & rotation              |



Fast containers are not magic.

They are the result of good design, smart tuning and continuous monitoring.





# DOCKER SECURITY INTERNALS

Security in Docker is about reducing risk,  
isolating workloads and protecting the host & kernel.



Containers share the host kernel.

Security = Isolation + Least Privilege + Controls.



Used by:

- Security Engineers
- DevOps Teams
- Platform Engineers
- Compliance & Audit

## 1. LINUX CAPABILITIES

- Linux capabilities break root privileges into small, specific permissions.
- Containers only get the capabilities they need.
- Default Docker profile drops dangerous caps.

Common Capabilities

| Capability           | What it Allows          | Risk if Misused |
|----------------------|-------------------------|-----------------|
| CAP_NET_ADMIN        | Network admin           | High            |
| CAP_SYS_ADMIN        | System admin            | Very High       |
| CAP_CHOWN            | Change file ownership   | Medium          |
| CAP_DAC_OVERRIDE     | Bypass file permissions | High            |
| CAP_NET_BIND_SERVICE | Bind to ports < 1024    | Low             |

Example

```
Drop all capabilities
and add only what
you need

docker run \
--cap-drop ALL \
--cap-add NET_BIND_SERVICE \
nginx:alpine
```

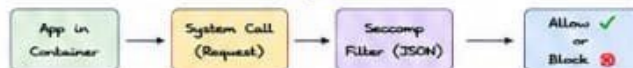


Principle:  
Give only what  
is required,  
Nothing more!

## 2. SECCOMP (SECURE COMPUTING MODE)

- Seccomp filters system calls that a container can make.
- Docker uses default seccomp profile to block dangerous syscalls.
- Prevents container escape via unsafe kernel calls.

Seccomp Flow



View default seccomp profile:

`docker run --rm alpine cat /usr/share/containers/seccomp.json | less`

Best Practice

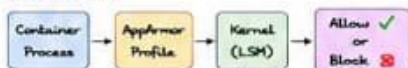
- ✓ Keep default seccomp profile unless you know what you're doing.
- ✓ Customize only when necessary.



## 3. APPARMOR

- AppArmor is a Linux security module.
- It enforces fine-grained profiles for applications.
- Useful on Ubuntu/Debian systems.

How it works



Commands

- `sudo aa-status` # Check AppArmor status
- `sudo aa-enforce /etc/apparmor.d/docker-default`
- `sudo aa-logprof` # Generate profile from logs

Example Profile

```
profile docker-default {
rules...
deny ptrace,
deny mount,
deny network raw,
...
}
```

## 4. SELINUX

- SELinux provides mandatory access control (MAC).
- Common on RHEL, CentOS, Fedora.
- Labels control what containers can access.

Check SELinux  
status  
# or  
getenforce

| Mode       | Description                  |
|------------|------------------------------|
| Enforcing  | Enforce policy (recommended) |
| Permissive | Log violations but allow     |
| Disabled   | No SELinux protection        |



Docker + SELinux Tips

- ✓ Use `:z` or `:Z` on volume mounts for proper labeling.
- ✓ Keep SELinux in enforcing mode in production.
- ✓ Audit denials: `ausearch -m avc -ts recent`

## 5. ROOTLESS RUNTIME

- Run containers without root on the host.
- Uses user namespaces + unprivileged ports.
- Reduces impact if a container is compromised.

Benefits

- ✓ No root daemon required
- ✓ Strong isolation from host
- ✓ Less attack surface
- ✓ Perfect for dev & CI



Run Rootless Docker

```
Install rootless mode
dockerd-rootless-setuptool.sh install

Start rootless daemon
systemctl --user start docker
```

Limitations

- ✗ Can't bind to port < 1024 without extra config
- ✗ Some features have limitations (e.g. iptables, overlays)

## 6. KERNEL PROTECTION & HARDENING

- Protect the host kernel — the most critical layer.
- Use namespaces, cgroups, LSMs and kernel settings.

Key Hardening Areas

| Area                | Best Practices                                         |
|---------------------|--------------------------------------------------------|
| Namespace Isolation | PID, Net, IPC, UTS, Mount, User namespaces             |
| Cgroups             | Limit CPU, Memory, I/O, PIDs, Network                  |
| Read-only Root FS   | Mount root FS as read-only                             |
| Drop Privileges     | Run as non-root user (USER in Dockerfile)              |
| Kernel Sysctls      | Tune sysctl (e.g. <code>net.ipv4.ip_forward=0</code> ) |
| Immutable Infra     | Use minimal base images (Alpine/Distroless)            |

## 7. SECURITY IN DEPTH: HOW IT ALL WORKS TOGETHER



Remember:

"Containers share the kernel, not the trust. Protect the kernel, secure everything."



## 8. COMMON SECURITY RISKS & HOW TO FIX

| Risk                    | Why It's Dangerous        | How to Fix                                             |
|-------------------------|---------------------------|--------------------------------------------------------|
| Running as Root         | Full control if exploited | ✓ Use USER nonroot<br>✓ Enable rootless runtime        |
| Excess Capabilities     | Too many privileges       | ✓ Drop all caps<br>✓ Add only required ones            |
| Unrestricted Networking | Access to host & internet | ✓ Use network policies<br>✓ Restrict egress            |
| Old / Vulnerable Images | Known CVEs                | ✓ Scan images (Trivy/Graype)<br>✓ Keep updated         |
| Secrets in Image        | Leaked credentials        | ✓ Use Docker secrets<br>✓ External vaults (e.g. Vault) |

## 9. USEFUL COMMANDS CHEAT SHEET

```
docker run --cap-drop ALL --cap-add NET_BIND_SERVICE nginx
docker run --security-opt seccomp=default.json alpine
docker run --security-opt apparmor=docker-default alpine
docker run --security-opt label-type=container-runtime-t alpine
docker run --read-only -v /tmp:/tmp alpine
docker inspect <container> | grep -i "security_opt"
docker scout cves <image>
sestatus / getenforce
aa-status
```

# Run with minimal caps  
# Use seccomp profile  
# Use AppArmor profile  
# SELinux label  
# Read-only root FS  
# Check security opts  
# Scan image for CVEs  
# Check SELinux status  
# Check AppArmor status



Security is not a feature. It's a habit.  
Build secure. Run secure. Stay secure.







## DOCKER DAEMON CONFIGURATION



The Docker daemon is highly configurable.

A small change can have a big impact.



The Docker daemon (dockerd) is controlled using a JSON configuration file:  
/etc/docker/daemon.json



Used by:

- DevOps Engineers
- Platform Engineers
- SREs
- System Administrators

## 1. DAEMON.JSON OVERVIEW

- The primary configuration file for dockerd.
- Located at /etc/docker/daemon.json
- JSON format.
- Changes require daemon restart.
- Run: systemctl restart docker

File Path

/etc/docker/daemon.json

Example (Full)

```
{
 "registry-mirrors": ["https://mirror.gcr.io", "https://dockerhub.aak8s.cn"],
 "dns": ["8.8.8.8", "1.1.1.1"],
 "log-driver": "json-file",
 "log-opt": {"max-size": "10m", "max-file": "3"},
 "storage-driver": "overlay2",
 "storage-opt": ["overlay2.override_kernel_check=true"],
 "default-runtime": "runc",
 "runtimes": {
 "runc": {"path": "runc", "runtimeArgs": []}
 },
 "live-restore": true,
 "insecure-registries": ["192.168.1.0/24"],
 "bip": "172.18.0.1/16"
}
```

## 2. REGISTRY MIRRORS

- Mirrors speed up image pulls.
- Reduces latency and Docker Hub rate limits.
- Specify one or more mirror URLs.

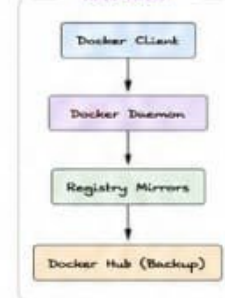
Example

```
{
 "registry-mirrors": [
 "https://mirror.gcr.io",
 "https://dockerhub.aak8s.cn",
 "https://hub-mirror.c.163.com"
]
}
```

Best Practice

- ☒ Use multiple mirrors.
- ☒ Place fastest mirror at the top.
- ☒ Always keep Docker Hub as fallback.

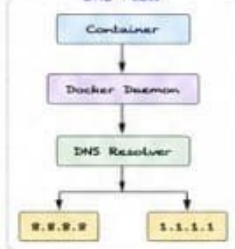
Pull Flow



## 3. DNS CONFIGURATION

- Defines DNS servers for containers.
- Overrides the host's DNS for containers.
- Improves name resolution reliability.

DNS Flow



Example

```
{
 "dns": ["8.8.8.8", "1.1.1.1"],
 "dns-search": ["corp.local"],
 "dns-opt": ["ndots:2", "timeout:2"]
}
```

Tips

- ☒ Use reliable public DNS.
- ☒ Add internal DNS first (if any).
- ☒ Use dns-search for internal domains.

## 4. LOGGING CONFIGURATION

- Configure the logging driver and options.
- Helps control log size and rotation.
- Prevents disk from filling up.

Example (json-file)

```
{
 "log-driver": "json-file",
 "log-opt": {
 "max-size": "10m",
 "max-file": "3"
 }
}
```

Best Practice

- ☒ Set max-size and max-file.
- ☒ Use centralized logging in production.

Common Log Drivers

| Driver    | Description                  |
|-----------|------------------------------|
| json-file | Default. Logs to JSON file.  |
| syslog    | Send logs to syslog server.  |
| journald  | Use systemd journal.         |
| gelf      | Send logs to Graylog.        |
| fluentd   | Send logs to Fluentd.        |
| awslogs   | Send logs to AWS CloudWatch. |

Monitor

- ☒ Check disk usage.
- ☒ Rotate and prune logs.

## 5. RUNTIME CONFIGURATION

- Choose default OCI runtime.
- Configure multiple runtimes (runc, crun, kata).
- Enables advanced runtime features.

Example

```
{
 "default-runtime": "runc",
 "runtimes": {
 "runc": {"path": "runc", "runtimeArgs": []},
 "crun": {"path": "crun", "runtimeArgs": []}
 }
}
```

Popular Runtimes

- runc - Default OCI runtime
- crun - Lightweight & fast
- kata - VM-based isolation
- gvisor - User-space kernel
- yakti - Rust-based runtime

Tip: Use lightweight runtimes for better performance.

## 6. STORAGE CONFIGURATION

- Defines storage driver and options.
- Critical for performance and stability.
- Improper storage settings can break containers.

Example (overlay2)

```
{
 "storage-driver": "overlay2",
 "storage-opt": [
 "overlay2.override_kernel_check=true"
]
}
```

Best Practice

- ☒ Use overlay2 on most Linux systems.
- ☒ Ensure backing filesystem is XFS or ext4.

Common Storage Drivers

- overlay2 - Recommended
- btrfs - Copy-on-write
- zfs - Advanced features
- devicemapper - Legacy
- xfs - Slow (not recommended)

Check Storage

- ☒ docker info | grep "Storage Driver"
- ☒ df -h | grep docker

## 7. OTHER USEFUL OPTIONS

| OPTION                   | DESCRIPTION                                    |
|--------------------------|------------------------------------------------|
| live-restore             | Keep containers running during daemon restart. |
| insecure-registries      | Allow communication with insecure registries.  |
| bip                      | Configure the default bridge IP.               |
| mtu                      | Set MTU for Docker networks.                   |
| max-concurrent-downloads | Limit concurrent image downloads.              |
| max-concurrent-uploads   | Limit concurrent upload layers.                |
| userland-proxy           | Enable/disable userland proxy.                 |
| iptables                 | Enable IPv6 networking.                        |
| experimental             | Enable experimental features.                  |

## 8. RELOAD &amp; VALIDATE CONFIGURATION

Reload Steps

1. Edit /etc/docker/daemon.json
2. Validate JSON syntax  
auto dockerd --validate
3. Restart Docker daemon  
auto systemctl restart docker
4. Verify  
docker info | less

Troubleshooting

- Invalid JSON will prevent Docker from starting.
- Check logs:  
journalctl -u docker.service -xe
- Use online JSON validator.
- Always backup daemon.json before making changes.



Good configuration = Stable, Secure and Fast Docker environment.

Tune the daemon. Monitor it. Trust it.





## DOCKER INTERNALS TROUBLESHOOTING

★ Systematic debugging starts with understanding how Docker works under the hood. ★



Know the layers. Follow the path. Use the right tools.  
Solve the root cause, not just the symptom.



Used by:

- DevOps Engineers
- SREs
- System Administrators
- Platform Engineers

## 1. DAEMON FAILURES

- Docker daemon (dockerd) not running or unresponsive.
- Causes: config errors, resource issues, crash loops.
- Check service status and logs first.
- Root cause is often in logs or configuration.

| Common Symptoms                     | Possible Causes                           |
|-------------------------------------|-------------------------------------------|
| Cannot connect to the Docker daemon | Daemon not running / socket not found     |
| Docker service fails to start       | Invalid daemon.json / permission issues   |
| High memory / CPU usage by dockerd  | Resource exhaustion / too many containers |
| Docker stuck on starting            | Corrupted storage / hanging container     |

## Diagnostic Commands

```
Check daemon status
sudo systemctl status docker

View logs
sudo journalctl -u docker -f

Check configuration
cat /etc/docker/daemon.json

Validate JSON syntax
sudo dockerd --validate
```

## Resolution Tips

- ✓ Fix configuration issues
- ✓ Ensure sufficient resources
- ✓ Restart the daemon
- ✓ Check disk space
- ✓ Review system logs

## 2. RUNTIME FAILURES

- Containers fail to start or exit unexpectedly.
- Check container logs, exit codes and runtime errors.
- Issues may be in image, config, app or dependencies.

| Common Symptoms             | Possible Causes                         |
|-----------------------------|-----------------------------------------|
| Container exits immediately | Application crash / wrong command       |
| OCI runtime error           | Image corruption / incompatible runtime |
| Healthcheck failing         | Service/port not ready / app error      |
| OOMKilled (out of memory)   | Memory limit too low / memory leak      |

## Comments

- `docker ps -a`
- `docker logs <container>`
- `docker inspect <container>`
- `docker events --since 10m`
- `docker top <container>`

## What to Check

- ✓ Exit code (`docker inspect`)
- ✓ Logs for error messages
- ✓ Resource limits (CPU/MEM)
- ✓ Volume mounts & permissions
- ✓ Healthcheck configuration

## 3. STORAGE DEBUGGING

- Storage issues affect images, containers and performance.
- Check disk usage, layer integrity, driver health.
- Common: no space left, corrupted layers, slow IO.

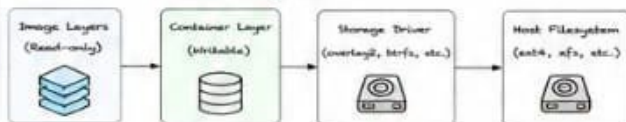
## Check Storage

```
docker system df
docker volume ls
docker image ls
df -h
du -sh /var/lib/docker/*
```

## Common Issues &amp; Fixes

- No space left → Prune unused data
- Slow performance → Check storage driver / disk IO
- Corrupted layer → Remove image & re-pull
- Volume issues → Check permissions / driver
- Build cache too big → `docker builder prune`

## Docker Storage Flow



Copy-on-write layers managed by storage driver

## 4. NETWORK DEBUGGING

- Containers cannot reach each other or external services.
- Check network configuration, DNS, routing and firewall.
- Use containers' network namespace for deep debug.

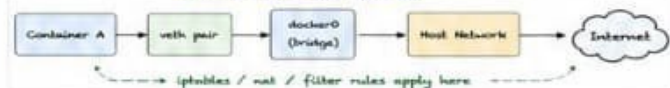
## Common Problems

- Cannot reach Internet
- Container cannot resolve DNS
- Port not accessible
- Inter-container communication fails

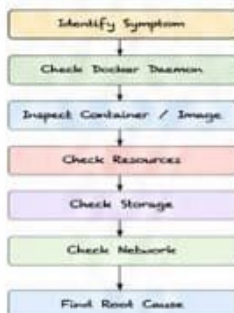
## Debug Commands

```
docker network ls
docker network inspect <network>
docker exec -it <ctr> ip a
docker exec -it <ctr> ping 8.8.8.8
docker exec -it <ctr> dig google.com
nsenter -t $(pidof <proc>) -n ip a
```

## Network Packet Path (Bridge)



## 5. DIAGNOSTIC WORKFLOW



## Key Questions

1. Is the daemon running?
2. Are there any error logs?
3. Does the container start?
4. Are resources sufficient?
5. Is storage healthy?
6. Is network working?
7. What changed recently?



**Golden Rule:**  
Always collect logs and inspect before taking action.

## 6. INTERNAL TROUBLESHOOTING TOOLS

| TOOL                              | PURPOSE                              | EXAMPLE USAGE                               |
|-----------------------------------|--------------------------------------|---------------------------------------------|
| <code>docker events</code>        | Real-time events from daemon         | <code>docker events --since 5m</code>       |
| <code>docker inspect</code>       | Deep inspection (JSON output)        | <code>docker inspect &lt;ctr&gt;</code>     |
| <code>nsenter</code>              | Enter container namespaces           | <code>nsenter -t \$(pid) -n bash</code>     |
| <code>ctr</code> (containerd CLI) | Manage containers via containerd     | <code>ctr --nri.k8s.io containers ls</code> |
| <code>crictl</code>               | CRI runtime CLI (K8s environments)   | <code>crictl ps -a</code>                   |
| <code>nerdctl</code>              | Docker-compatible CLI for containerd | <code>nerdctl ps</code>                     |
| <code>strace</code>               | Trace system calls                   | <code>strace -p \$(pid)</code>              |
| <code>lsuf</code>                 | List open files / sockets            | <code>lsuf -p \$(pid)</code>                |
| <code>iptables / nft</code>       | Inspect firewall & NAT rules         | <code>iptables -t nat -L -n --</code>       |
| <code>bpftool</code>              | Inspect eBPF programs/maps           | <code>bpftool prog show</code>              |
| <code>journalctl</code>           | System and service logs              | <code>journalctl -u docker</code>           |

## HANDY ONE-LINERS

- Restart Docker daemon  
`sudo systemctl restart docker`
- Clean up everything  
`docker system prune -a --volumes`
- Free space quickly  
`docker builder prune -a`
- Check open files for Docker  
`lsuf | grep docker`

## LOG LOCATIONS

- Docker daemon logs:  
`/var/log/docker.log` (if configured)
- Systemd logs:  
`journalctl -u docker`
- Container logs:  
`/var/lib/docker/containers/*/*-json.log`

## BEST PRACTICES

- ✓ Monitor daemon and containers
- ✓ Set resource limits
- ✓ Use dedicated storage
- ✓ Keep Docker and OS updated
- ✓ Automate log collection
- ✓ Test changes in staging first



Observe → Inspect → Isolate → Fix → Verify  
Understand Docker internals. Troubleshoot with confidence.





# ADVANCED DOCKER ARCHITECTURE INTERVIEW QUESTIONS

★ 50 High-Impact Questions with Model Answers ★



Understand the architecture. Explain it clearly.  
Solve problems like an engineer.



### Legend

- OCI = Open Container Initiative
- RT = Runtime
- ST = Storage
- KR = Kernel

## 50 ARCHITECTURE INTERVIEW QUESTIONS (WITH MODEL ANSWERS)

### 1. OCI QUESTIONS (10)

1. What is OCI?  
A: Open Container Initiative defines open standards for container formats and runtime.
2. What are the two main OCI specifications?  
A: Image Spec (image format) and Runtime Spec (container runtime & lifecycle).
3. What is the role of config.json in an OCI image?  
A: It defines container runtime configuration (env, cmd, mounts, etc.).
4. How are layers defined in OCI image spec?  
A: As tar archives with a diff\_id and chain\_id for integrity.
5. What is an OCI runtime?  
A: A runtime that implements OCI Runtime Spec (e.g., runc, crun).
6. What is the purpose of the OCI runtime spec?  
A: To standardize how a container is started, stopped and managed.
7. How does an OCI runtime receive the configuration?  
A: Through a JSON bundle containing config.json and rootfs.
8. What is an OCI image manifest?  
A: A JSON file that references config and layer digests.
9. What is the difference between manifest and manifest list?  
A: Manifest list (index) points to manifests for multiple platforms.
10. How does content addressability work in OCI?  
A: Objects are stored and referenced by their digest (sha256).

### 3. STORAGE QUESTIONS (10)

21. What is a Docker storage driver?  
A: A component that manages read-write layers for containers.
22. How does overlay2 storage driver work?  
A: Uses OverlayFS with lowerdir (image layers) and upperdir (container write layer).
23. What are copy-on-write layers?  
A: Layers are shared and only changed files are copied to writable layer.
24. Where are Docker layers stored?  
A: `/var/lib/docker/overlay2/ids/`
25. How does Docker ensure layer integrity?  
A: By using content-addressable storage (sha256 digests).
26. What is the difference between overlay2 and aufs?  
A: overlay2 uses native OverlayFS (in kernel) and is faster and stable.
27. How does Docker clean up unused layers?  
A: Through garbage collection (prune) based on reference count.
28. What is the role of snapshotter in containerd?  
A: It manages filesystem snapshots for containers.
29. What is a whiteout file?  
A: A special file that marks a file as deleted in a layer.
30. How does Docker handle disk full situations?  
A: It fails writes; clean unused images/containers; monitor disk usage.

### 5. ADVANCED ARCHITECTURE QUESTIONS (10)

41. How does Docker networking work internally?  
A: Uses network namespaces, with pairs, bridge (vethers) and iptables.
42. What is user-defined bridge network?  
A: Custom bridge with isolated subnet and DNS.
43. How does Docker DNS work?  
A: Embedded DNS server (127.0.0.11) for container name resolution.
44. How does overlay networking work in Swarm/K8s?  
A: VLAN tunnels overlay traffic between nodes.
45. How does Docker Builds enable multi-platform builds?  
A: Uses QEMU emulation and BuildKit with manifest lists.
46. How does Docker ensure image security?  
A: Image signing (Notary/Sign), scanning, least privilege runtime.
47. How does Docker handle large-scale deployments?  
A: Swarm/K8s orchestration, registries, caching, monitoring.
48. What happens when a container restarts?  
A: Containerd + runc recreate namespaces, cgroups and mount points.
49. How does Docker handle logs and metrics?  
A: json-file/driver logging and Collector/Prometheus for metrics.
50. How would you troubleshoot a container start failure?  
A: Check logs → runtime error → config → resources → dependencies

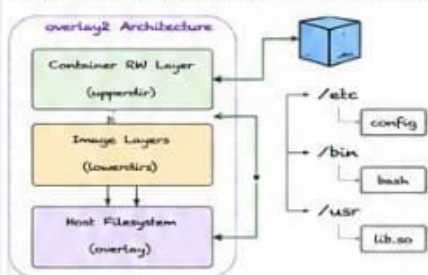
## 2. RUNTIME QUESTIONS (10)

11. What is the role of containerd in Docker?  
A: It manages container lifecycle, images, snapshots and plugins.
12. How does containerd interact with runc?  
A: containerd uses runc as the low-level OCI runtime to create containers.
13. What are the main components of containerd?  
A: client, daemon, content store, metadata store, tasks, snapshots, plugins.
14. What is a shim in containerd?  
A: A lightweight process between containerd and the actual runtime.
15. What is the container lifecycle in containerd?  
A: Create → Start → Running → Stop → Delete.
16. What is runc?  
A: A CLI tool that implements the OCI Runtime Spec.
17. How does runc create a container?  
A: By setting up namespaces, cgroups, rootfs and starting init.
18. What is the purpose of an init process in a container?  
A: PID 1 handles signals and reaps zombie processes.
19. What is the difference between runc and runC?  
A: runC is the original name; now commonly written as runc.
20. How can you inspect a running container at runtime level?  
A: Use nsenter, runc exec, or inspect namespaces and cgroups.

#### 4. KERNEL QUESTIONS (10)

31. What is a Linux namespace?  
A: A kernel feature that isolates system resources (PID, NET, IPC, etc.).
32. Which namespaces are used by Docker?  
A: PID, NET, IPC, MNT, UTS, USER, CGROUP.
33. What is a cgroup?  
A: A kernel feature to limit, account and isolate resource usage.
34. How does Docker limit memory?  
A: Using cgroups (memory.limit\_in\_bytes, memory.max).
35. How does Docker limit CPU?  
A: Using cgroups (cpu.shares, cpu.cfs\_quota\_us).
36. What is the role of capabilities in Docker?  
A: Fine-grained permissions to avoid running as full root.
37. What is seccomp?  
A: A kernel feature to filter syscalls for security.
38. How does Docker implement seccomp?  
A: Using seccomp profiles (default.json) to whitelist syscalls.
39. What is AppArmor/SELinux in Docker?  
A: LSMs that enforce security policies for containers.
40. What is the difference between rootful and rootless at kernel level?  
A: Rootless uses user namespaces and unprivileged kernel features.

SAMPLE MODEL ANSWER (Q: 22 - How overlay2 works?)



- Overlayd uses Linux OverlayFS.
- Lower layers are read-only image layers.
- Upper layer is writable (copy-on-write).
- Only changed files are stored in upperdir.
- This saves storage and improves performance.

## Klasa Poziorna:

- ✓ Fast and stable (internal routine)
- ✓ Supports hard links, ch\_type, long paths
- ✓ Best for production workloads

### Useful Commands

```
docker info | grep Storage Driver
docker system df
ls -l /var/lib/docker/overlay2/
```



### Interview Success Tips

- ✓ Understand the architecture end-to-end.
- ✓ Explain with diagrams and examples.
- ✓ Relate concepts to real production problems.
- ✓ Practice explaining out loud.



Remember

It's not about memorizing.  
It's about understanding  
how Docker works  
under the hood.



### Your Superpower

Deep knowledge + Clear explanations  
= Architect level engineer.  
Keep learning. Keep building.





## DOCKER INTERNALS CHEAT SHEET

★ One Page. Everything You Need. Keep This Handy! ★

Master the internals. Use the right command,  
inspect deeply, and solve faster!

## Legend

|     |                  |
|-----|------------------|
| CTR | = containerd CLI |
| BX  | = Builder        |
| RT  | = Runtime        |
| ST  | = Storage        |
| NW  | = Networking     |

## 1. CTR COMMANDS (containerd CLI)

| COMMAND                              | DESCRIPTION                             |
|--------------------------------------|-----------------------------------------|
| ctr version                          | Show containerd client & server version |
| ctr namespaces ls                    | List namespaces                         |
| ctr images ls                        | List images                             |
| ctr images pull <image>              | Pull image                              |
| ctr images inspect <image>           | Inspect image                           |
| ctr containers ls                    | List containers                         |
| ctr containers create <image> <name> | Create container                        |
| ctr containers start <id>            | Start container                         |
| ctr containers pause <id>            | Pause container                         |
| ctr containers delete <id>           | Delete container                        |
| ctr snapshots ls                     | List snapshots                          |
| ctr content ls                       | List content                            |
| ctr events                           | Show real-time events                   |

## 2. BUILDX COMMANDS

| COMMAND                                                                       | DESCRIPTION            |
|-------------------------------------------------------------------------------|------------------------|
| docker buildx version                                                         | Show Buildx version    |
| docker buildx ls                                                              | List builder instances |
| docker buildx create --name mybuilder                                         | Create new builder     |
| docker buildx use mybuilder                                                   | Switch builder         |
| docker buildx inspect mybuilder                                               | Inspect builder        |
| docker buildx rm mybuilder                                                    | Remove builder         |
| docker buildx build -t alpine:latest .                                        | Build image (classic)  |
| docker buildx build --platform linux/amd64,linux/arm64 -t app:multi .         | Multi-platform build   |
| docker buildx build --push .                                                  | Build & push           |
| docker buildx build --cache-to type=registry,ref=repo/cache,mode=max --push . | Export cache           |
| docker buildx build --cache-from type=registry,ref=repo/cache .               | Import cache           |

## 3. DAEMON.JSON OPTIONS

| OPTION                     | PURPOSE                                       |
|----------------------------|-----------------------------------------------|
| "data-root"                | Docker data directory                         |
| "hosts"                    | Daemon listen address                         |
| "log-driver"               | Default logging driver                        |
| "log-opts"                 | Logging options                               |
| "storage-driver"           | Storage driver                                |
| "registry-mirrors"         | Registry mirrors                              |
| "dns"                      | Default DNS servers                           |
| "dns-search"               | DNS search domains                            |
| "bip"                      | Default bridge IP                             |
| "fixed-cidr"               | Default subnet for bridge                     |
| "default-runtime"          | Default OCI runtime                           |
| "runtimes"                 | Configure additional runtimes                 |
| "max-concurrent-downloads" | Full concurrency                              |
| "ipv6"                     | Enable IPv6                                   |
| "live-restore"             | Keep containers running during daemon restart |
| "userland-proxy"           | Enable userland proxy for published ports     |

## Example daemon.json

```
{
 "data-root": "/var/lib/docker",
 "storage-driver": "overlay2",
 "registry-mirrors": ["https://mirror.gcr.io"],
 "log-driver": "json-file",
 "log-opts": {"max-size": "10m", "max-file": "3"},
 "dns": ["8.8.8.8", "1.1.1.1"],
 "default-runtime": "runc",
 "live-restore": true
}
```

## 4. RUNTIME COMMANDS

| COMMAND                 | DESCRIPTION               |
|-------------------------|---------------------------|
| docker run <image>      | Run container             |
| docker run -d <image>   | Run in detached mode      |
| docker ps               | List running containers   |
| docker ps -a            | List all containers       |
| docker inspect <id>     | Inspect container (JSON)  |
| docker logs <id>        | View logs                 |
| docker exec -it <id> sh | Exec in running container |
| docker stop <id>        | Stop container            |
| docker start <id>       | Start container           |
| docker rm <id>          | Remove container          |
| docker rm -f <id>       | Force remove              |
| docker top <id>         | List running processes    |
| docker stats            | Live container stats      |
| docker events           | Stream events             |
| docker info             | System-wide information   |
| docker version          | Client/Server versions    |
| docker system df        | Disk usage summary        |
| docker system prune     | Clean up unused data      |

## 5. STORAGE COMMANDS

| COMMAND                      | DESCRIPTION                  |
|------------------------------|------------------------------|
| docker image ls              | List images                  |
| docker image inspect <image> | Inspect image metadata       |
| docker history <image>       | Show image layers history    |
| docker rmi <image>           | Remove image                 |
| docker image prune           | Remove dangling images       |
| docker volume ls             | List volumes                 |
| docker volume inspect <id>   | Inspect volume               |
| docker volume rm <id>        | Remove volume                |
| docker volume prune          | Remove unused volumes        |
| docker system df -v          | Detailed disk usage          |
| docker build --squash .      | Squash layers (experimental) |
| docker save <image> --imgtar | Export image to tar          |
| docker load -i imgtar        | Import image from tar        |

## 6. NETWORKING COMMANDS

| COMMAND                                                  | DESCRIPTION                  |
|----------------------------------------------------------|------------------------------|
| docker network ls                                        | List networks                |
| docker network inspect <net>                             | Inspect network              |
| docker network create <net>                              | Create bridge network        |
| docker network rm <net>                                  | Remove network               |
| docker network connect <net> <id>                        | Connect container to network |
| docker network disconnect <net> <id>                     | Disconnect container         |
| docker network prune                                     | Remove unused networks       |
| docker run --network <net> <image>                       | Run with specific network    |
| docker exec <id> ip a                                    | Show interfaces in container |
| docker exec <id> ip route                                | Show routes                  |
| docker exec <id> ip netns exec <id> cat /etc/resolv.conf | View DNS config              |
| docker inspect <id>   grep -A 30 "Networks"              | See network settings         |

## 7. QUICK REFERENCE TABLES

## COMMON DEFAULT PATHS

| PATH                       | PURPOSE                   |
|----------------------------|---------------------------|
| /var/lib/docker            | Docker data root          |
| /var/lib/docker/overlay2   | Overlay2 storage          |
| /var/lib/docker/containers | Container metadata & logs |
| /var/run/docker.sock       | Docker daemon socket      |
| /etc/docker/daemon.json    | Daemon configuration      |
| /var/log/docker.log        | Docker daemon logs        |

## RUNTIME COMPARISON

| RUNTIME                 | TYPE    | NOTES                        |
|-------------------------|---------|------------------------------|
| runc                    | OCI     | Default, stable, widely used |
| containerd-shim-runc-v2 | Shim    | Manages container lifecycle  |
| crun                    | OCI     | Faster, written in C         |
| kata-runtime            | VM      | Extra isolation (VM based)   |
| gvisor                  | Sandbox | Userspace kernel (runc)      |

## DOCKER NETWORK DRIVERS

| DRIVER  | USE CASE                       |
|---------|--------------------------------|
| bridge  | Default single-host networking |
| host    | No isolation, use host network |
| none    | No networking                  |
| overlay | Multi-host (Swarm/K8s)         |
| macvlan | Containers get own MAC on LAN  |
| ipvlan  | L2/L3 mode for scale           |

## STORAGE DRIVERS COMPARISON

| DRIVER         | PERF  | FEATURES               | BEST FOR                |
|----------------|-------|------------------------|-------------------------|
| overlay2       | ★★★★☆ | Stable, copy-on-write  | General purpose (Linux) |
| fuse-overlayfs | ★★★★☆ | Rootless friendly      | Rootless containers     |
| trfs           | ★★★★☆ | Snapshots, quotas      | Advanced users          |
| zfs            | ★★★★☆ | Snapshots, compression | High performance        |
| devicemapper   | ★★★☆☆ | Thin provisioning      | Legacy environments     |

## OCI LAYERS (IMAGE)

| LAYER      | DESCRIPTION              |
|------------|--------------------------|
| Base Image | FROM layer               |
| Layer 1..N | RUN / COPY / ADD layers  |
| Top Layer  | Writable container layer |
| Union FS   | Merged (overlay2, etc.)  |

## PORT PUBLISHING MODES

| MODE      | EXAMPLE              | DESCRIPTION             |
|-----------|----------------------|-------------------------|
| Host Port | -p 8080:80           | Map host:container port |
| IP + Port | -p 127.0.0.1:8080:80 | Bind to specific IP     |
| UDP       | -p 53:53/udp         | Publish UDP port        |
| Range     | -p 8000-8005:80      | Map port range          |

## INSPECTION SHORTCUTS

- docker inspect <id>
- docker inspect <image>
- docker inspect <network>
- docker inspect <volume>



## EVENT TYPES (examples)

- create, start, stop, die
- pull, push
- connect, disconnect
- rename, destroy

## DAEMON SIGNALS

- kill -HUP <pid> # reload config
- kill -USR1 <pid> # log reopen
- kill -USR2 <pid> # debug dump

## PRO TIPS

- Use --format in inspect for clean output.
- Prefer overlay2 for best performance.
- Enable live-restore in production.
- Always monitor logs and events.

Know the internals. Use the tools. Solve anything.  
Docker is just Linux + namespaces + cgroups + union FS.





## DOCKER ARCHITECTURE MASTER CHECKLIST

Your Complete Mastery Roadmap to Docker Internals



From the kernel to the cloud.

From one container to thousands.

Master every layer. Build with confidence. Operate like a pro.

This is your Docker Mastery Checklist.



## Legend

- ☒ = I Know Well  
☐ = I Understand  
☐ = Need More Practice  
☐ = Don't Know Yet

## 1. ENGINE (DOCKER DAEMON)



- ☐ Understand Docker daemon process
- ☐ Know daemon architecture
- ☐ Configure daemon.json
- ☐ Manage registries & mirrors
- ☐ Configure logging drivers
- ☐ Configure DNS settings
- ☐ Understand runtime selection
- ☐ Configure storage drivers
- ☐ Manage system resources
- ☐ Reload & validate daemon config

## Key Outcome:

You can configure, tune and troubleshoot the Docker Engine like an expert.

## 2. RUNTIME (CONTAINER RUNTIME)



- ☐ Understand OCI runtime spec
- ☐ Know runc architecture
- ☐ Understand container lifecycle
- ☐ Explore container namespaces
- ☐ Understand cgroups & limits
- ☐ Understand capabilities
- ☐ Configure seccomp, AppArmor, SELinux
- ☐ Rootless runtime concepts
- ☐ Debug runtime failures
- ☐ Use ctr, runc, nri tools

## Key Outcome:

You understand how containers run at the OS level.

## 3. STORAGE (PERSISTENCE)



- ☐ Understand image layers
- ☐ Understand union filesystems
- ☐ Manage storage drivers
- ☐ Understand copy-on-write
- ☐ Manage volumes (local, bind)
- ☐ Understand volume lifecycle
- ☐ Understand containerd snapshotter
- ☐ Prune & cleanup storage
- ☐ Import / Export images
- ☐ Optimize image layers

## Key Outcome:

You can manage, optimize and troubleshoot Docker storage confidently.

## 4. NETWORKING



- ☐ Understand network namespaces
- ☐ Understand veth pairs
- ☐ Understand Linux bridge
- ☐ Packet flow in Docker
- ☐ Understand embedded DNS
- ☐ Configure DNS & host resolution
- ☐ Overlay networking (VXLAN)
- ☐ Multi-host networking
- ☐ Inspect & debug networks
- ☐ Secure container networking

## Key Outcome:

You can design, inspect and troubleshoot Docker networks at any scale.

## 5. SECURITY



- ☐ Understand Linux capabilities
- ☐ Configure seccomp profiles
- ☐ Understand AppArmor
- ☐ Understand SELinux
- ☐ Rootless mode best practices
- ☐ User & group mapping
- ☐ Image scanning basics
- ☐ Content trust & signatures
- ☐ Secrets management
- ☐ Apply least privilege principle

## Key Outcome:

You can secure containers from kernel to runtime to registry.

## 6. PERFORMANCE



- ☐ Understand container startup
- ☐ Optimize image size
- ☐ Optimize build cache
- ☐ Understand memory management
- ☐ Configure CPU limits & shares
- ☐ Understand I/O & disk performance
- ☐ Use multi-stage builds
- ☐ Use .dockerignore effectively
- ☐ Monitor container performance
- ☐ Tune runtime & daemon

## Key Outcome:

You can build and run high-performance containerized applications.

## 7. BUILD (IMAGES &amp; BUILDX)



- ☐ Understand Docker build process
- ☐ Use Dockerfile best practices
- ☐ Use BuildKit features
- ☐ Use buildx & builder instances
- ☐ Multi-platform builds (AMD64/ARM64)
- ☐ Cache export / import
- ☐ Cross compilation strategies
- ☐ Manifest lists & image index
- ☐ Push images to registries
- ☐ SBOM & provenance (basics)

## Key Outcome:

You can build, optimize and publish multi-platform production images.

## 8. TROUBLESHOOTING



- ☐ Diagnose daemon failures
- ☐ Diagnose runtime failures
- ☐ Storage debugging
- ☐ Network debugging
- ☐ Use diagnostic workflow
- ☐ Use internal troubleshooting tools
- ☐ Read & analyze logs
- ☐ Inspect containers deeply
- ☐ Monitor resources & events
- ☐ Solve root causes, not symptoms

## Key Outcome:

You can troubleshoot any Docker issue methodically and fast.

## MASTER CHECKLIST (THE COMPLETE REVIEW)

Mark your level honestly. Aim for all ☒ before moving to production at scale!

| DOMAIN          | I KNOW WELL<br>(☑)       | I UNDERSTAND<br>(□)      | PRACTICE MORE<br>(△)     | DON'T KNOW<br>(X)        | NOTES |
|-----------------|--------------------------|--------------------------|--------------------------|--------------------------|-------|
| Engine          | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> |       |
| Runtime         | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> |       |
| Storage         | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> |       |
| Networking      | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> |       |
| Security        | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> |       |
| Performance     | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> |       |
| Build           | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> |       |
| Troubleshooting | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> |       |

## Mastery Goal:

All ☒ in "I Know Well" column  
→ You are Docker Internals Master!



## FINAL READINESS ASSESSMENT

Self Assessment: Rate Yourself (1 = Beginner, 5 = Expert)

|                                        |                             |
|----------------------------------------|-----------------------------|
| Understand Docker Engine Internals     | <input type="checkbox"/> /5 |
| Understand Container Runtime Internals | <input type="checkbox"/> /5 |
| Understand Storage Internals           | <input type="checkbox"/> /5 |
| Understand Networking Internals        | <input type="checkbox"/> /5 |
| Understand Security Internals          | <input type="checkbox"/> /5 |
| Understand Performance & Optimization  | <input type="checkbox"/> /5 |
| Build & Multi-platform expertise       | <input type="checkbox"/> /5 |
| Troubleshooting & Debugging skills     | <input type="checkbox"/> /5 |

TOTAL SCORE

☐ /40

## What Can You Do With Confidence?

- ☐ Design Docker environments from scratch
- ☐ Configure & secure Docker for production
- ☐ Optimize performance & reduce costs
- ☐ Build & publish multi-platform images
- ☐ Troubleshoot complex issues
- ☐ Handle high availability
- ☐ Scale with best practices
- ☐ Mentor others

YOU  
MADE  
IT!

Mastery is not about knowing everything.  
It's about understanding deeply and applying perfectly.  
Keep learning. Keep building. Keep shipping.  
Docker is your tool. Architecture is your superpower.



## Pro Tip

- ☐ Practice daily
- ☐ Break things safely
- ☐ Read docs
- ☐ Build real projects
- ☐ Teach others

